

University of Bremen

Bachelor's Thesis

Analysis of the Prevalence and Vulnerability of Qt Deserialization in Open-Source Projects

Paul Bode | 6188386

First Supervisor: PD Dr. Karsten Sohr
Second Supervisor: Prof. Dr. Ute Bormann

University of Bremen | Faculty 3: Mathematics and Computer Science
Submission date: August 18, 2026
Publication date: November 1, 2026

Abstract

QDataStream is a Qt class for binary serialization that the framework itself discourages using with untrusted data. This bachelor's thesis analyzes the prevalence and vulnerability of QDataStream in open-source Qt projects. GitHub is searched to identify 69,763 verified Qt projects of which 7,221 are found to use QDataStream. The QDataStream projects are ranked by a popularity metric to allow for prioritization in the vulnerability analysis. Three weaknesses are identified that, in combination, could introduce two vulnerabilities into a project using QDataStream. A large language model is used to write static analysis checks to detect these weaknesses. A static analysis of the 100 highest-ranked projects from the prevalence analysis, using the static analysis tool Joern, indicates that 75 out of the 97 analyzed projects contain at least one of the weaknesses. In 51 of the 97 analyzed projects, indicators for a combination of two weaknesses that could introduce a vulnerability are found. In two of the projects analyzed, a case study identified a vulnerability that impairs availability and can be exploited by a local attacker. These results are used to discuss the risks of using QDataStream and possible ways to reduce them. The static-analysis checks are made freely available to support future research and to let developers examine their own projects.

Note on this version

This is the published version of the thesis. Certain parts of the original submission, namely the acknowledgments and the formal university documentation, have been redacted. In addition, the links to the thesis repository have been updated to point to its current location. In agreement with the affected projects (Krita, Tiled, and Qt), the 90-day disclosure period described in the thesis was shortened, so this version is published earlier than originally planned. The scientific content is otherwise unchanged from the submitted version.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Research Target and Scope . . . 1
- 1.3 Structure of This Thesis . . . 2

2 Basics **3**
- 2.1 Definitions . . . 3
- 2.2 Serialization and Deserialization . . . 4
- 2.3 Deserialization in the Qt Framework . . . 4
- 2.4 Vulnerability of QDataStream . . . 5
- 2.5 Summary . . . 6

3 Prevalence Analysis **7**
- 3.1 Methodology . . . 7
  - 3.1.1 Data Source . . . 7
  - 3.1.2 Searching for QDataStream . . . 8
  - 3.1.3 Ranking . . . 9
  - 3.1.4 Pipeline Implementation in Python . . . 10
- 3.2 Results . . . 12
- 3.3 Prevalence Analysis Discussion . . . 16

4 Vulnerability Analysis **19**
- 4.1 Methodology . . . 19
  - 4.1.1 QDataStream Weaknesses . . . 19
  - 4.1.2 Analyzing Source Code . . . 22
  - 4.1.3 Static Analysis Tool Selection . . . 24
  - 4.1.4 Large Language Models for Code Analysis . . . 24
  - 4.1.5 Our Approach . . . 25
- 4.2 Results . . . 27
  - 4.2.1 Check Verification . . . 31
  - 4.2.2 Static Analysis . . . 31
  - 4.2.3 Case Study . . . 32
- 4.3 Vulnerability Analysis Discussion . . . 36

5 Discussion **39**
- 5.1 Interpretation of the Results . . . 39
- 5.2 Threats to Validity and Limitations . . . 40

6 Conclusion **43**
- 6.1 Answers to the Research Questions . . . 43
- 6.2 Overall Conclusion . . . 44

7 Ethics Considerations **45**

Use of AI-Based Applications **47**

References **51**

Lists **53**

List of Figures 53

List of Tables 53

List of Listings 53

List of Abbreviations 54

1. Introduction

1.1 Motivation

Software libraries are a cornerstone of modern software development. They provide a foundation for reusing software components, thereby making software development more efficient and cost-effective. Important functionality that is used by many developers can be worked on in a centralized fashion. This is especially clear in the case of open-source software libraries. Here the source code of a library is publicly available under certain rules, listed in an open-source license. Corporations and individual contributors drive the development of these libraries, often without direct financial compensation. However, there is an inherent risk in using software libraries. Software products become dependent upon the libraries they use, which in turn can have dependencies themselves. The cascade of these dependencies can be viewed as a software supply chain. Buggy or malicious code in a library can cause damage across a multitude of software products that depend on it.[1]

A popular example of a library to analyze these risks is the Qt framework (Qt). Qt allows for cross-platform development of C++, QML or Python applications and is used by over a billion devices and applications, according to the Qt Group.[2] In a prior report, Hüther et al. (2023) of the software engineering working group at the University of Bremen already found multiple weaknesses in different components of the Qt framework. Based on this, Huth (2026) wrote a bachelor's thesis, analyzing a specific component of the library more precisely. This component, the class QDataStream, can be used to serialize and deserialize binary data. Huth showed that in some cases of QDataStream usage, security risks are possible.

1.2 Research Target and Scope

The goal of this thesis is the analysis of the impact of QDataStream security risks on open-source Qt projects. The scope is limited to projects written in C++ because the potential security risks discussed by Huth (2026, pp. 13–24) were demonstrated using C++ code. This thesis is split into two parts.

The prevalence analysis in the first part will investigate which open-source projects are using the QDataStream class and therefore are potentially vulnerable. Additionally, the projects using QDataStream are ranked by a popularity metric for prioritization.

The vulnerability analysis in the second part identifies two vulnerabilities arising from the use of QDataStream. It then analyzes the 100 highest-ranked projects from the prevalence analysis to identify indications of these vulnerabilities in the current usage of QDataStream.

[1] `https://arcticwolf.com/resources/blog/log4j-retrospective/`, last accessed on 03.05.2026
[2] `https://www.qt.io/development/qt-framework`, last accessed on 30.05.2026

This leads to the following four research questions:

- How many open-source projects that use the Qt framework and are written in C++ make use of the class QDataStream? (**RQ1**)
- Which QDataStream weaknesses discussed by Hüther et al. (2023) and Huth (2026) could introduce a vulnerability into a Qt project? (**RQ2**)
- How many of the 100 highest-ranked projects from the prevalence analysis show indications of vulnerabilities introduced by use of QDataStream? (**RQ3**)
- Does an indication reported by the analysis correspond to an actual vulnerability in real Qt projects? (**RQ4**)

Beyond answering these questions, this thesis contributes a set of static analysis checks for the weaknesses discussed. They are made freely available so that developers can examine their own Qt projects, not only the ones analyzed here. Throughout this thesis, we use the term checks to refer collectively to the rules by which analysis tools detect weaknesses, since different tools call them rules, checks, or queries.

1.3 Structure of This Thesis

The second chapter provides the basics of serialization, both in general and in the context of the Qt framework. With the help of a real-world example, the possible risk of using QDataStream is illustrated.

In the third chapter, this thesis identifies open-source projects that use the QDataStream class. This is done by searching GitHub for Qt projects and locally scanning them for QDataStream. The resulting projects are ranked with a metric based on popularity to provide a priority list for the vulnerability analysis.

The fourth chapter addresses the vulnerability analysis. It first defines three weaknesses, identified by Huth and Hüther et al., that can lead to two vulnerabilities in code using QDataStream. It then discusses how source code can be analyzed for these weaknesses, selects a suitable static analysis tool, and describes how a large language model is used to generate the corresponding analysis checks. These checks are used to analyze the 100 highest-ranked projects from the prevalence analysis. Based on these findings, a case study identifies a vulnerability in a Qt component that affects multiple projects.

The fifth chapter discusses the results of both analyses and relates them to the work by Huth and Hüther et al. It also addresses threats to validity and limitations of the approach.

The sixth chapter answers the four research questions and draws an overall conclusion. It closes with an outlook on future work.

Finally, the seventh chapter discusses the ethical considerations of this thesis, including the reasoning behind providing all artifacts, such as source code, checks, and analysis results, in the thesis repository.[3]

[3] `https://github.com/paul-bode/bachelors-thesis`

2. Basics

This chapter provides a basic understanding of what serialization and deserialization are and gives an overview of how they work in the Qt framework. It first defines important terms and concepts of security engineering and vulnerability management, and then illustrates QDataStream deserialization in both a basic and a real-world vulnerability example. Basic knowledge of the security objectives of confidentiality, integrity, and availability is assumed.

2.1 Definitions

This thesis uses the following definitions for fundamental security engineering terminology. The first important term is vulnerability. According to Anderson (2020, p. 15), a vulnerability "is a property of a system or its environment which, in conjunction with an internal or external threat, can lead to a security failure". In the context of this thesis, internal or external threat refers to an attacker with malicious intent, and a security failure refers to an impairment of the security objectives of confidentiality, integrity, or availability. Distinct from a vulnerability, the term weakness will be used to describe "a condition in a software, firmware, hardware, or service component that, under certain circumstances, could contribute to the introduction of vulnerabilities".[1] This means, a weakness can, but does not have to introduce a vulnerability. Another important term is exploit, which will refer to the means of attacking a vulnerability, for example a malicious file that gets deserialized.

There is a triad of important tools when dealing with weaknesses and vulnerabilities. The Common Weakness Enumeration (CWE) system offers a list of weaknesses together with a classification taxonomy to describe them.[2] The Common Vulnerabilities and Exposures (CVE) program on the other hand helps to "identify, define, and catalog publicly disclosed cybersecurity vulnerabilities".[3] Vulnerabilities are given identifiers called CVEs, for example CVE-2026-25166. At last, the Common Vulnerability Scoring System (CVSS) is a standardized way to score the severity of a vulnerability.[4] These three in conjunction allow researchers and practitioners to specify, categorize and prioritize vulnerabilities.

Software is not just a technical but also a political topic. This thesis uses both the terms free software and open-source software, sometimes in conjunction as free and open-source software. There are many similarities but also slight differences between these two philosophies that fall outside the scope of this work.[5] This thesis is only concerned with the characteristic of inspectable source code that both share. More information can be found at the Open Source Initiative[6] and the Free Software Foundation[7] respectively.

[1] `https://cwe.mitre.org/about/faq.html#what_is_cwe_weakness`, last accessed on 04.05.2026
[2] `https://cwe.mitre.org/about/index.html`, last accessed on 04.05.2026
[3] `https://www.cve.org/About/Overview`, last accessed on 04.05.2026
[4] `https://www.first.org/cvss/`, last accessed on 04.05.2026
[5] `https://www.gnu.org/philosophy/open-source-misses-the-point.html`, last accessed on 29.05.2026
[6] `https://opensource.org/osd`, last accessed on 29.05.2026
[7] `https://www.fsf.org/`, last accessed on 29.05.2026

2.2 Serialization and Deserialization

Software applications often use objects at runtime to model and organize functionality. The process of serialization converts the state of an object from memory into a format that can be persisted or exchanged. The resulting format can either be binary (stream of bytes) or text-based (stream of characters) (Grochowski et al. 2019, p. 2). Examples of formats are JavaScript Object Notation (JSON) and Extensible Markup Language (XML) for text-based cases or Concise Binary Object Representation (CBOR) and Protocol Buffers (Protobuf) on the binary side.

Deserialization is the counterpart to serialization. Given a binary or text-based input, the process recreates the original object (Grochowski et al. 2019, p. 1). Depending on the given application, the input could, for example, originate from a file or be received over a network. This makes deserialization prone to facilitating bugs and vulnerabilities. Input from outside the application's influence could be tampered with by an attacker, leading to unintended or malicious behavior. Deserializing input that is not trusted or sanitized, called tainted input, can lead to denial-of-service, access control, or remote code execution attacks.[8] This is a significant problem, as demonstrated by a number of recent CVEs in products such as Windows (CVE-2025-21171 and CVE-2026-25166, both CVSS severity: high), MLflow (CVE-2024-37052, CVSS severity: high), and Kibana (CVE-2024-37288, CVSS severity: critical).[9] To detect such vulnerabilities, taint analysis tracks the flow of data from attacker-controlled input points (sources) to sensitive parts of the program where this data could cause unintended behavior (sinks) (Schwartz et al. 2010, p. 320). Such an analysis is therefore referred to as a source-to-sink analysis.

Serialization and deserialization are also referred to as marshaling and unmarshaling (Grochowski et al. 2019, p. 1). This thesis uses the terms serialization and deserialization throughout.

2.3 Deserialization in the Qt Framework

The Qt framework is a software library developed by the Qt Group. It is split into different modules ranging from core business logic, networking, and file access to user interface and graphics. Officially supported programming languages are QML, Python and C++.[10] Qt allows cross-platform development targeting desktop, mobile, microcontrollers and embedded devices.[11]

The Qt framework has support for different types of serialization and thus deserialization. Text-based options are JSON and XML while CBOR and QDataStream handle the binary operations. This thesis will concentrate on the QDataStream class because Huth (2026) already identified security risks in it.

[8] `https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html`, last accessed on 05.05.2026

[9] All CVSS base scores are taken from the NVD records, available at `https://nvd.nist.gov/vuln/detail/<CVE-ID>`, last accessed on 05.05.2026

[10] `https://www.qt.io/development/qt-framework/languages`, last accessed on 05.05.2026

[11] `https://www.qt.io/development/qt-framework/platforms`, last accessed on 05.05.2026

QDataStream provides a host-system-independent stream of binary data, allowing parsing of data between different operating systems and CPU architectures. It supports both big-endian and little-endian byte order. QDataStream is used to read and write binary data to and from a QIODevice. The latter being a parent class for different types of input and output classes like QFile, QBluetoothSocket and many more.[12] The following examples from the Qt documentation show how QDataStream is used inside C++ code.

Listing 2.1 creates a file and serializes two different types of objects into it. As shown, QDataStream overloads the operators << and >> in order to serialize and deserialize data for the different classes.

C++

```
QFile file("file.dat");
file.open(QIODevice::WriteOnly);
QDataStream out(&file);   // we will serialize the data into the file
out << QString("the answer is");   // serialize a string
out << (qint32)42;        // serialize an integer
```

Listing 2.1: Write Binary Data to a Stream.

Listing 2.2 shows the opposite direction. The program opens the file and QDataStream deserializes the two fields from it.

C++

```
QFile file("file.dat");
file.open(QIODevice::ReadOnly);
QDataStream in(&file);    // read the data serialized from the file
QString str;
qint32 a;
in >> str >> a;           // extract "the answer is" and 42
```

Listing 2.2: Read Binary Data From a Stream.

2.4 Vulnerability of QDataStream

The vulnerability of QDataStream is not just theoretical. One example of this class being exploitable is shown in CVE-2018-1000178.[13] The vulnerable application Quassel is an Internet Relay Chat (IRC) application. The IRC protocol uses the client-server model and allows for clients to exchange text-based messages (*Internet Relay Chat Protocol* 1993, pp. 3–6). In version 0.12.4 of Quassel, the following lines of code potentially allow an attacker to execute code remotely. The root of this problem lies in the way `datastreampeer.cpp`[14] deserializes an incoming message.

[12] `https://doc.qt.io/qt-6/qdatastream.html`, last accessed on 05.05.2026
[13] `https://nvd.nist.gov/vuln/detail/CVE-2018-1000178`, last accessed on 06.05.2026
[14] `https://github.com/quassel/quassel/blob/0.12.4/src/common/protocols/datastream/datastreampeer.cpp`, last accessed on 06.05.2026

```C++
56 void DataStreamPeer::processMessage(const QByteArray &msg)
57 {
58     QDataStream stream(msg);
59     stream.setVersion(QDataStream::Qt_4_2);
60     QVariantList list;
61     stream >> list;
62     if (stream.status() != QDataStream::Ok) {
63         close("Peer sent corrupt data, closing down!");
64         return;
65     }
66     //...
67 }
```

Listing 2.3: Quassel 0.12.4 – datastreampeer.cpp.

Line 61 uses the >> operator to deserialize an entire QVariantList from tainted, potentially attacker-controlled, bytes (Listing 2.3). To mitigate this problem in version 0.12.5, Quassel developers wrote a custom function to deserialize the message stream. The function `deserialize` is overloaded for different classes of incoming data and checks them accordingly. It replaces the vulnerable » operator in line 61 (Listing 2.4).

```C++
56 void DataStreamPeer::processMessage(const QByteArray &msg)
57 {
58     QDataStream stream(msg);
59     stream.setVersion(QDataStream::Qt_4_2);
60     QVariantList list;
61     if (!Serializers::deserialize(stream, list))
62         close("Peer sent corrupt data, closing down!");
63     if (stream.status() != QDataStream::Ok) {
64         close("Peer sent corrupt data, closing down!");
65         return;
66     }
67     //...
68 }
```

Listing 2.4: Quassel 0.12.5 – datastreampeer.cpp.

2.5 Summary

As described above, the nature of deserialization and the case of QDataStream in particular result in a number of challenges when using it. Furthermore, the Qt framework itself discourages usage of QDataStream in "security-sensitive situations".[15] In combination, this calls for an analysis of the prevalence and vulnerability of QDataStream in open-source projects.

[15] `https://doc.qt.io/qt-6/qdatastream.html#corruption-and-security`, last accessed on 06.05.2026

3. Prevalence Analysis

The goal of this chapter is to analyze the prevalence of the QDataStream class in open-source projects that use the Qt framework. To achieve this, we have to find Qt projects and identify the ones using QDataStream. The results also need to be ranked to enable prioritization for the vulnerability analysis in the next chapter. Finally, the findings are used to address the first research question: How many open-source projects that use the Qt framework and are written in C++ make use of the class QDataStream?

3.1 Methodology

3.1.1 Data Source

Open-source software is a global phenomenon built on collaborative development platforms that offer features like version control, bug tracking, and means for developers to communicate. There are many providers of platforms for hosting source code of open-source projects. Table 3.1 shows a list of the most popular providers with estimates for the number of projects and users hosted on them. This is by no means an exhaustive list, and the estimates should be treated with caution. The platforms are not transparent about how they count these metrics, and alternative numbers exist.

Name	**Projects**	**Users**
GitHub[a]	$> 420,000,000$	$> 180,000,000$
GitLab[b,c]	$\gg 546,000$	$> 50,000,000$
Assembla[d]	$> 2,000,000$	$> 1,500,000$
Codeberg[e]	$593,961$	$363,610$
SourceForge[f]	$> 500,000$	$< 20,000,000$
Launchpad[g,h]	$48,965$	$9,256,855$

[a] `https://github.com/about`, last accessed on 12.06.2026.
[b] `https://web.archive.org/web/20190918034231/https://about.gitlab.com/2016/02/11/gitlab-retrospective/`, last accessed on 12.06.2026.
[c] `https://about.gitlab.com/`, last accessed on 12.06.2026.
[d] `https://get.assembla.com/`, last accessed on 12.06.2026.
[e] `https://codeberg.org/about`, last accessed on 12.06.2026.
[f] `https://sourceforge.net/about`, last accessed on 12.06.2026.
[g] `https://launchpad.net/projects`, last accessed on 12.06.2026.
[h] `https://launchpad.net/people`, last accessed on 12.06.2026.

Table 3.1: Source Code Hosting Providers.

The table shows that GitHub is by far the largest provider of source code hosting. GitHub also has two mature and full-fledged application programming interfaces (APIs), allowing for automated and reproducible requests. To keep the attributes of the analyzed projects comparable and to increase the reproducibility of the analysis, GitHub was chosen as the only data source.

3.1.2 Searching for QDataStream

There are multiple ways to query the data on GitHub for Qt projects and the QDataStream class. Because we want the ability to automate the analysis, the most promising methods are BigQuery, the GitHub REST API and the GitHub GraphQL API.

BigQuery is an "autonomous data and AI platform" from Google that among other things allows running Structured Query Language (SQL) queries against large datasets.[1] Some datasets are publicly available and one of these is *GitHub Activity Data*, containing a "full snapshot of the content of more than 2.8 million open source GitHub repositories".[2] The advantage of BigQuery is the ability to both search for Qt projects and identify those that use QDataStream in a single step. This is possible because the source code can be queried directly. The disadvantage is that the data is significantly outdated; according to the dataset information, it was last updated in 2022. Such a big gap between data snapshot and reality is not compatible with the aim of this thesis. Projects found to be using QDataStream in 2022 could have already removed it by now, severely diminishing the value of our results. This makes BigQuery not suitable for this analysis.

The remaining two options, the GitHub REST API[3] and the GitHub GraphQL API[4], work directly with the current data hosted by GitHub. While both are suitable for executing the search query proposed later, GraphQL allows requesting specific and nested data with a single request. An example of nested data is a "pull request along with its commits, non-review comments, and reviews".[5] This allows us to only request the data needed, reducing resource consumption and making it convenient for collecting the data for the ranking of the projects described in the next section.

In order to get real-time data on Qt projects and search for QDataStream, we use the following approach. First, we call GitHub's GraphQL API with an encoded search query to find Qt projects. The search query allows us to specify exactly what we are searching for, by passing the search term `qt` and the programming language `C++`. To improve the quality of the results we exclude mirror, template, and archive repositories. These three restrictions are important because we are interested only in real-world projects, hosted directly on GitHub, and projects that are actively used and developed and would thus benefit from a vulnerability analysis. This leads to the following query,

```
qt language:cpp mirror:false template:false archived:false,
```

resulting in a list of possible Qt projects.

We then analyze each project locally by cloning the source code from GitHub to a local git repository and checking two conditions. First, does the project's code contain further evidence that this really is a Qt project? We do this by evaluating whether certain file name and file content indicators connected to Qt are present in the repository. This helps us to

[1] `https://cloud.google.com/bigquery`, last accessed on 29.05.2026

[2] `https://console.cloud.google.com/marketplace/product/github/github-repos`, last accessed on 29.05.2026

[3] `https://docs.github.com/en/rest/about-the-rest-api/about-the-rest-api`, last accessed on 29.05.2026

[4] `https://docs.github.com/en/graphql/overview/about-the-graphql-api`, last accessed on 29.05.2026

[5] `https://docs.github.com/en/graphql/guides/migrating-from-rest-to-graphql#example-nesting`, last accessed on 12.06.2026

reduce false positives from the search query. Second, all C++ source and header files are searched for the term `QDataStream`.

3.1.3 Ranking

Although the prevalence analysis addresses a research question on its own, it also provides the basis for the vulnerability analysis in the next chapter. Early into the research for this thesis, we conducted a preliminary analysis of the number of projects using QDataStream, which indicated a significant usage. The large number of projects in scope indicates that analyzing all projects found to be using QDataStream for vulnerabilities is not feasible. In order to prioritize the further analysis of the projects, there needs to be a way to decide which projects to check for vulnerable QDataStream usage first. It follows that we need to rank the projects, preferably by calculating a score using API-requestable properties of the projects as input values.

The literature contains a wealth of research on ranking software projects. One example is the "Weighted Total Popularity Score" (WTPS) proposed by Al-Rubaye et al. (2020, p. 219), which takes the temporal growth of GitHub stars and forks into consideration. Tools like GitRank by Hasabnis (2022, p. 729) combine popularity with quality and maintainability metrics. This leads all the way to whole frameworks that include over 20 metrics (Singh et al. 2024, pp. 8–10).

One metric frequently used in these papers is popularity, measured through properties such as stars, forks, or watchers. For our ranking, this makes sense as well, because we assume that popular projects expose a larger number of potential victims to a vulnerability. Considering the over 70,000 projects in scope[6], the number of input values for calculating the score cannot be too high or the nested data too complex. This would result in high API request complexity, potentially triggering GitHub's API rate limit and preventing us from effectively calculating the score for this number of projects. Another restriction is the time frame imposed by the scope of a bachelor's thesis. The time available makes it difficult to obtain meaningful long-term trend data, leaving us with a single snapshot and ruling out scores like the WTPS.

The score used in this thesis is thus a subset of the popularity metrics used by Al-Rubaye et al. as well as Hasabnis. Properties used as input values are stars, forks, and watchers. Stars indicate that a user likes a project or finds it interesting, while watchers are users receiving notifications about changes made to the project.[7] Forks, on the other hand are copies of the projects repository indicating active interest in the source code and maybe even the willingness to contribute to it.[8] Together, these allow us to take both more passive (stars, watchers) and more active popularity (forks) into account. Stars also seem to play a role in the behavior of users: Borges and Tulio Valente (2018, p. 117) report that "three out of four developers consider the number of stars before using or contributing to GitHub projects".

[6] `https://github.com/search?q=qt+language%3Acpp+mirror%3Afalse+template%3Afalse+archived%3Afalse&type=repositories`, last accessed on 01.06.2026

[7] `https://github.blog/news-insights/notifications-stars/`, last accessed on 01.06.2026

[8] `https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo`, last accessed on 01.06.2026

One problem with this approach is that older projects, which had decades to accumulate popularity, can easily outrank new and very popular projects. In order to normalize the popularity of a project with its age, we can look at a field with similar concerns, the ranking of academic papers. We reviewed two metrics for time normalization from the literature and decided to use the function shown in Equation 3.1 with the value $\rho = 0.3$ proposed by Zhang et al. (2022, p. 4053). Here, $T_{\text{historical}}$ is the year of the analysis and t is the project's creation year.

$$Z_t = e^{-\rho \cdot (T_{\text{historical}} - t)} \tag{3.1}$$

We ruled out the "age- and field-normalized metrics" proposed by Vaccario et al. (2017, pp. 770–771) because the projects do not have clear attribution to distinct fields.

One idea that was discarded in the course of this thesis was to analyze keywords from the projects' description and labels for indicators of criticality. Terms like *browser*, *cryptocurrency* or *medical* would increase the score of a project, while keywords like *game* or *test* would decrease it. We discarded this approach because not all projects use a description or labels and choosing which words to include in these lists and how to weight them would be biased and arbitrary in comparison to the rest of the score.

This gives the popularity score p as a weighted sum of stars, forks, and watchers. The time normalization Z_t then scales it into the final score p_{norm}, as shown in Equation 3.2.

$$\begin{aligned} p &= 0.5 \cdot \text{stars} + 0.35 \cdot \text{forks} + 0.15 \cdot \text{watchers} \\ p_{\text{norm}} &= Z_t \cdot p \end{aligned} \tag{3.2}$$

We assigned the largest weight to stars and the second largest to forks, as these showed the highest and second-highest correlations with the WTPS respectively (Al-Rubaye et al. 2020, p. 221). For watchers, there was no correlation data, leading to a reduced weight.

This scoring system's limitations lie in the missing temporal data resolution. We compensate for this by aiming for a general correlation with the WTPS, which makes it suitable for the needs of this thesis.

3.1.4 Pipeline Implementation in Python

With data source, search query and ranking system in place, we use Python to combine these into the four-step pipeline shown in Figure 3.1. In the first step, **Query**, we use the search query to create a list of possible Qt repositories and the properties needed to clone and rank them. In the second step, **Scan**, we clone every repository on the list, verify whether they indeed use Qt and search for QDataStream. To make this fast and efficient, we avoid downloading full repository histories, restrict each checkout to the relevant Qt and C++ files, and process multiple projects in parallel.[9]

[9] Implemented using blobless clones with sparse checkouts; parallelism is handled by Python workers.

The approach to verify whether a project is using Qt is shown in Listing 3.1. We use `ripgrep`[10] to first search for Qt project files and, if none are found, for common Qt includes and identifiers in C++ files.

Python

```
QT_FILE_INDICATORS = ["*.pro","*.qml"]
QT_CODE_INDICATORS = [
    "#include <Qt",
    "#include <Q",
    "Q_OBJECT",
    "QApplication",
    "QCoreApplication",
    "QObject",
]
#...
def is_qt_repo(repo_path: str) -> bool:
    """Return True when the cloned repo looks like a Qt project."""

    file_globs = [arg for p in QT_FILE_INDICATORS for arg in ("-g", p)]
    rc, out, _ = run_cmd(
        [RG_PATH, "--files", "--no-messages", "--max-results", "1"] + file_globs
        + ["."],
        cwd=repo_path,
    )
    if rc == 0 and out.strip():
        return True

    pattern = "|".join(QT_CODE_INDICATORS)
    rc, out, _ = run_cmd([
        RG_PATH,
        "--files-with-matches",
        "--no-messages",
        "--type", "cpp",
        "--max-count", "1",
        "-e", pattern,
        ".",
    ], cwd=repo_path)

    if rc == 0 and out.strip():
        return True

    return False
```

Listing 3.1: Verifying Qt Projects.

For the verified Qt projects, `ripgrep` is used again to search for QDataStream. This results in a list of verified Qt projects and a subset list of projects using QDataStream. The third step, **Rank**, takes the list of QDataStream projects, calculates the score (Equation 3.2), and ranks the projects based on it. Finally, the last step, **Evaluate**, extracts the data necessary to answer the first research question and visualizes it. For each step, the full Python script can be found in the thesis repository.[11]

[10] `https://ripgrep.dev/`, last accessed on 19.06.2026
[11] `https://github.com/paul-bode/bachelors-thesis`

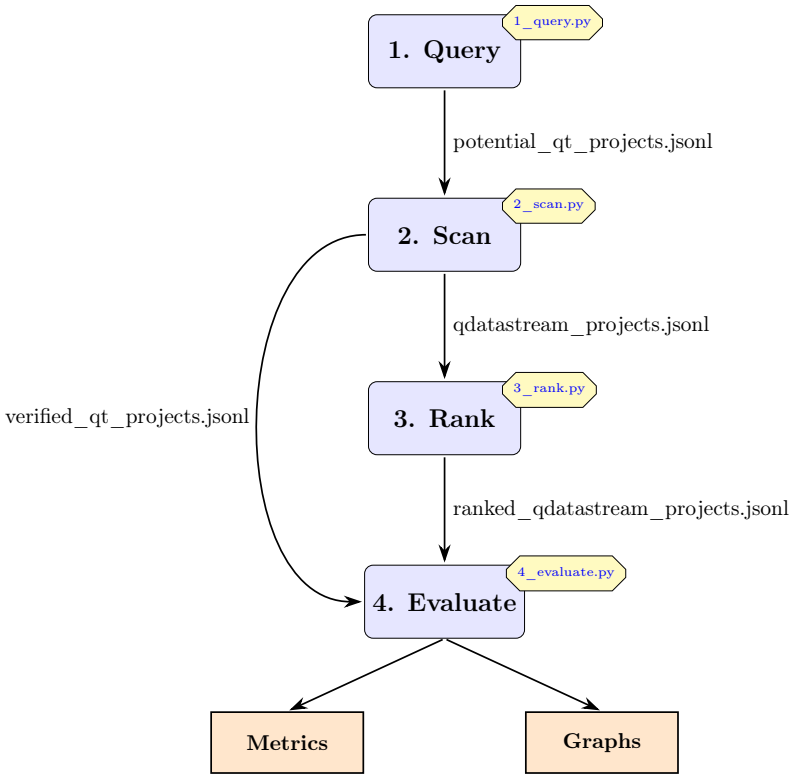


Figure 3.1: Prevalence Analysis Pipeline.

3.2 Results

The analysis took place in early June 2026, and included projects created between February 1, 2008, and May 31, 2026, covering 18 years and 4 months. We chose February 2008 as a starting point because the earliest public projects on GitHub are from that month.[12] Executing the search query resulted in 71,941 potential Qt projects, as shown in Figure 3.2. While scanning them, we identified 2,178 projects that showed no further evidence of being a Qt project, leading to a false-positive rate of $\approx 3.0275\%$. Out of the resulting 69,763 verified Qt projects, 7,221 used QDataStream. This results in a prevalence rate of $\approx 10.3508\%$, as summarized in Table 3.2.

As a preview for the vulnerability analysis, Table 3.3 lists the ten QDataStream projects with the highest scores, making them the most popular projects by our metric. These are part of the 100 projects that will be examined in the vulnerability analysis. The distribution depicted in Figure 3.3 shows that only a few projects using QDataStream have very high popularity scores, while the majority scores near zero. This suggests that a cutoff at the 100 highest-ranked projects covers most of the projects with substantial popularity, while the remaining ones score considerably lower.

[12] `https://github.com/search?q=is%3Apublic&type=repositories&s=updated&o=asc`, last accessed on 01.06.2026

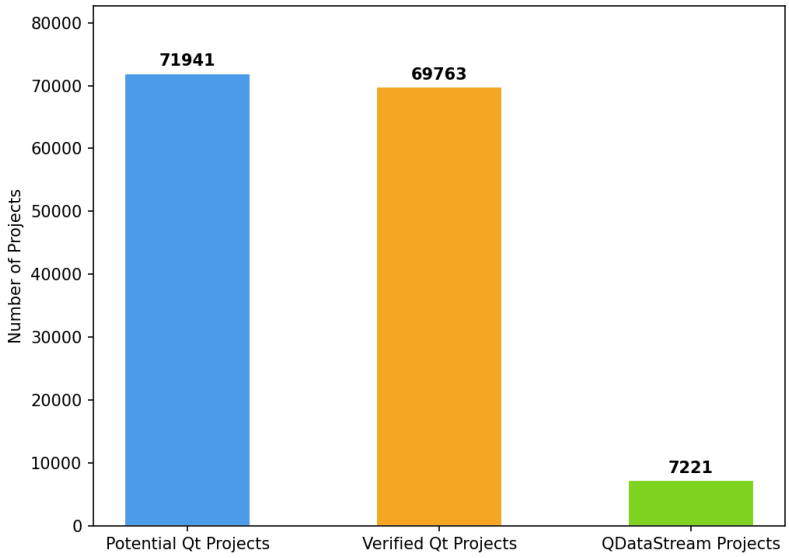


Figure 3.2: Prevalence Analysis Results.

Category	Count	Rate
Potential Qt Projects	71,941	
– False Positives	2,178	≈ 3.0275%
Verified Qt Projects	69,763	
QDataStream Projects	7,221	≈ 10.3508%

Table 3.2: Prevalence Analysis Results.

Rank	Project	Score
1	`Fincept-Corporation/FinceptTerminal`	8190.62950
2	`vicinaehq/vicinae`	3053.98607
3	`PrismLauncher/PrismLauncher`	1601.90142
4	`Qv2ray/Qv2ray`	1178.97375
5	`flameshot-org/flameshot`	1062.05208
6	`zhuzichu520/FluentUI`	1012.76502
7	`dail8859/NotepadNext`	914.78626
8	`quickshell-mirror/quickshell`	736.83450
9	`Serial-Studio/Serial-Studio`	642.59943
10	`feiyangqingyun/QWidgetDemo`	499.76917

Table 3.3: Top 10 QDataStream Projects Ranked by Score.

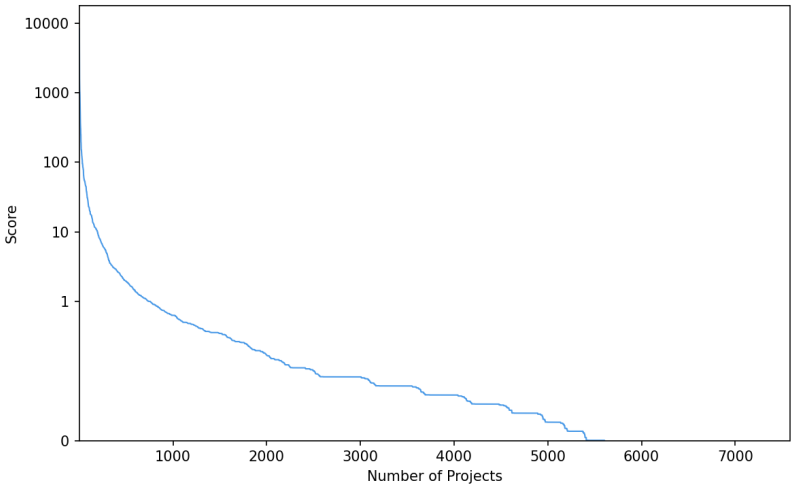


Figure 3.3: Score Distribution of QDataStream Projects.

Figure 3.4 compares the age distribution of Qt projects with the age distribution of projects using QDataStream. While there is a general upward trend in new Qt projects, the absolute number of projects adopting QDataStream appears to have stagnated. Observing the usage rate of QDataStream over the years, we suspected a linear relationship between the usage rate and the age of the project, and applied linear regression to quantify this relationship. The output of the `linregress`[13] function includes the slope of the regression line and the Pearson correlation coefficient. The Pearson correlation coefficient r is the covariance normalized by the standard deviations and measures the strength of the linear relationship between two variables. It lies "between -1 and $+1$, and if there is no correlation r equals zero" (Wohlin et al. 2024, p. 158). A value close to $+1$ indicates a strong positive linear relationship, while a value close to -1 indicates a strong negative one (Montgomery et al. 2010, Chapter 5.2). This approach follows prior work by Borges, Hora, et al. (2016, pp. 337–338) and Al-Rubaye et al. (2020, p. 218), who applied regression lines and the Pearson correlation coefficient to show correlation in their analyses. As shown by the regression line in Figure 3.5, older projects tend to use QDataStream more often, which is confirmed by a Pearson correlation coefficient of ≈ 0.862, indicating a strong positive linear relationship.

We also wanted to check whether our time normalization approach worked. Figure 3.6 visualizes the relationship between age and score of a project. The high scores on the left side of the graph indicate a slight bias to younger projects. This is supported by the negative slope of ≈ -0.4628, though the Pearson correlation coefficient of ≈ -0.017 indicates that the relationship is extremely weak. The correlation shows that our normalization approach has not entirely eliminated the influence of age on the score. The slight bias towards younger projects that our approach introduced may partially compensate for the inherent time advantage older projects have for collecting stars, forks, and watchers. An evaluation of this lies outside the scope of this thesis.

[13] `https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.linregress.html`, last accessed on 15.06.2026

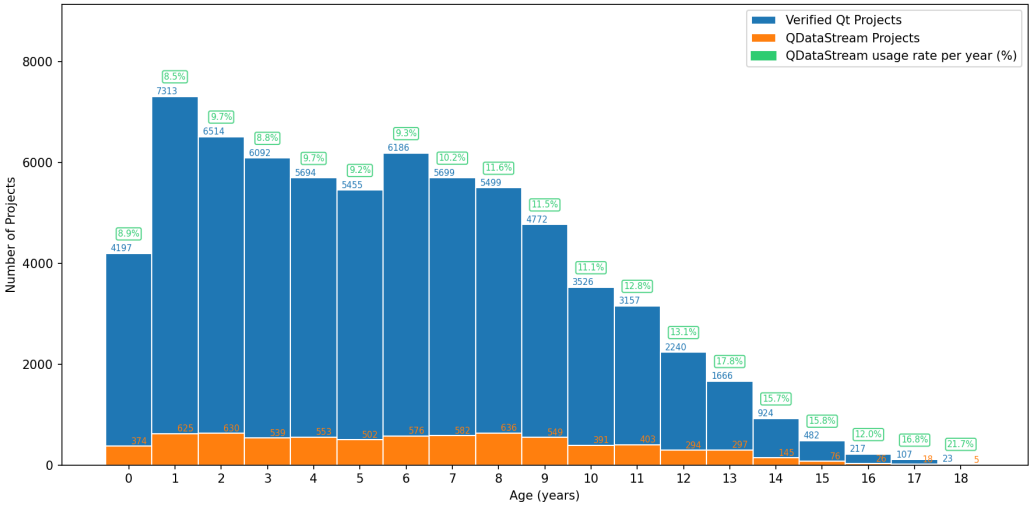


Figure 3.4: Age Distribution Comparison of Verified Qt Projects and QDataStream Projects.

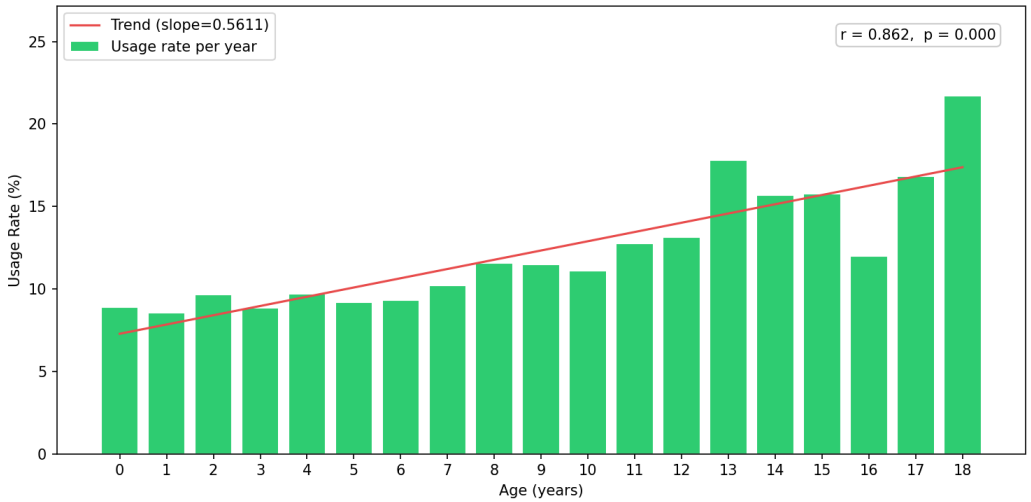


Figure 3.5: QDataStream Usage Rate per Year.

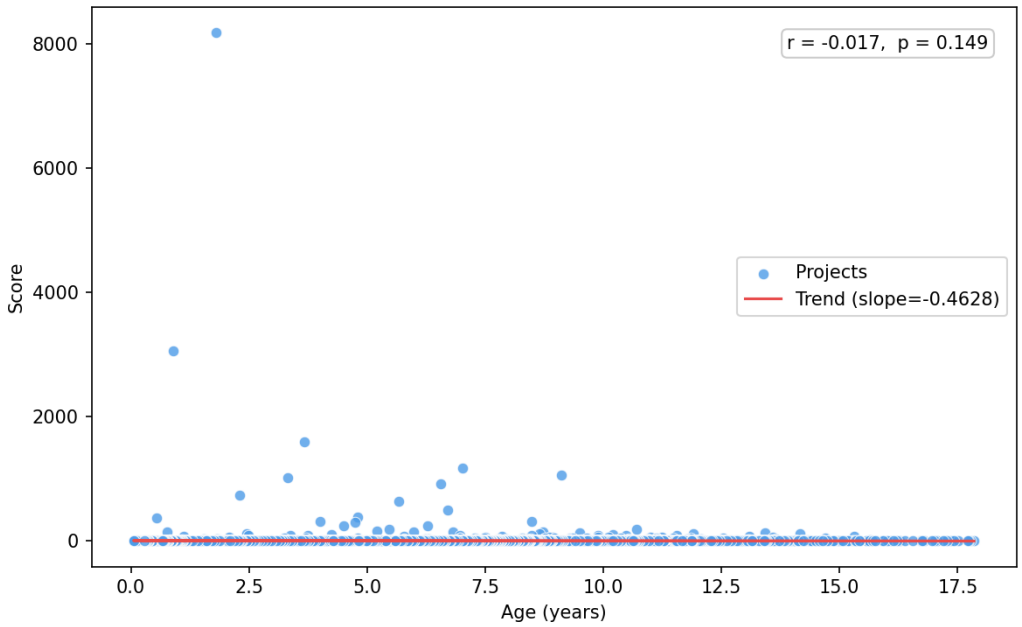


Figure 3.6: Relationship Between Score and Age in QDataStream Projects.

In summary, the results of this analysis allow us to answer the first research question.

RQ1: How many open-source projects that use the Qt framework and are written in C++ make use of the class QDataStream?

From our sample of 69,763 verified Qt projects, 7,221 used QDataStream. The QDataStream class thus has a prevalence rate of $\approx 10.3508\%$. Additionally, older projects are more likely to use QDataStream than younger projects.

3.3 Prevalence Analysis Discussion

Despite our best efforts, the prevalence analysis has three main limitations.

As mentioned above, the age normalization introduced a slight bias towards younger projects. This should be evaluated before the scoring formula is used in future analyses. One possible approach would be to compare different normalization formulas and iterate on their parameters. The output should be compared to the resulting age and score correlation and corrected to minimize the residual age bias.

The ranking system is not flawless either and the ranking of different projects is debatable. The third-highest ranked project is `PrismLauncher`[14], an application to manage multiple installations of the popular video game Minecraft. In the 21st place there is `Nextcloud Desktop`[15], a client for synchronizing Nextcloud data to a computer. While the first ranks higher by stars and forks, the second arguably handles more sensitive data and is also potentially used in critical corporate environments. There needs to be further discussion whether the ranking method is suitable for this type of analysis. Ranking based solely on popularity

[14] `https://github.com/PrismLauncher/PrismLauncher`, last accessed on 01.06.2026
[15] `https://github.com/nextcloud/desktop`, last accessed on 01.06.2026

appears to be insufficient. To address this shortcoming, future research should find a better way to incorporate criticality into such a ranking.

The biggest limitation is the way we search for Qt projects in the first place. Using our query with the GitHub search function results in many projects hosted on GitHub that use Qt, but not all of them. There is no way for us to evaluate how many projects we missed. Even more striking is that we might have missed the most important ones. One example we found manually is `Bitcoin Core`[16], which uses QDataStream in `paymentserver.cpp`[17], among other files. This project, being the backbone of the largest existing cryptocurrency by market capitalization[18], is both highly popular and critical. To partially address this limitation, we will add Bitcoin Core to the projects analyzed in the vulnerability analysis. A future iteration of the analysis would need to find a better way to identify Qt projects on GitHub or add more data sources.

Apart from these limitations, the analysis produced the data needed to answer the research question. While this was not the goal of the analysis, we additionally found a correlation between the age of the project and its usage of QDataStream. It would be interesting to find out why this is the case and what this means for further research.

A notable strength of this thesis's approach is the customizability of the pipeline we implemented. In their report, Hüther et al. (2023) identified a number of potentially vulnerable Qt classes and functions. Our toolchain could be reconfigured to search for classes and functions other than QDataStream by changing the search term.

Finally, the approach and results of this analysis also raise ethical questions. The tool we built is in essence a concatenation of scripts that takes a component of Qt and returns projects using it. An attacker could enter the name of a vulnerable component and obtain a list of popular targets. The University of Bremen has an ethics committee that assesses ethical aspects of research involving human subjects.[19] Because this thesis does not focus on humans and only uses publicly available data, the committee is not responsible in this case. Nevertheless, ethical considerations remain relevant. Many security research papers face the same ethical questions, and security conferences and symposia have therefore established ethical guidelines and policies. After reviewing those of the USENIX Security Symposium[20], the IEEE Symposium on Security and Privacy[21], and Network and Distributed System Security Symposium[22], we decided to address all moral implications of this thesis in Chapter 7.

[16] `https://github.com/bitcoin/bitcoin/tree/master`, last accessed on 01.06.2026
[17] `https://github.com/bitcoin/bitcoin/blob/master/src/qt/paymentserver.cpp`, last accessed on 01.06.2026
[18] `https://coinmarketcap.com/charts/bitcoin-dominance/`, last accessed on 29.06.2026
[19] `https://www.uni-bremen.de/rechtsstelle/ethikkommission`, last accessed on 22.06.2026
[20] `https://secartifacts.github.io/usenixsec2026/ethics-open-science`, last accessed on 22.06.2026
[21] `https://sp2027.ieee-security.org/cfpapers.html`, last accessed on 22.06.2026
[22] `https://www.ndss-symposium.org/ndss2027/submissions/call-for-papers/`, last accessed on 22.06.2026

4. Vulnerability Analysis

The goal of this chapter is to analyze the potential vulnerabilities that using QDataStream could introduce into open-source Qt projects, to determine how many projects show indications of them, and to verify whether such an indication corresponds to an actual vulnerability. To achieve this, we first review the weaknesses introduced by QDataStream to answer the second research question: Which QDataStream weaknesses discussed by Hüther et al. (2023) and Huth (2026) could introduce a vulnerability into a Qt project? Based on these findings, we then use a large language model to create static analysis checks. We apply these checks to the 100 highest-ranked projects from the prevalence analysis. The results answer the third research question: How many of the 100 highest-ranked projects from the prevalence analysis show indications of vulnerabilities introduced by use of QDataStream? Manually analyzing the projects allows us to answer the fourth research question: Does an indication reported by the analysis correspond to an actual vulnerability in real Qt projects?

4.1 Methodology

In this section, we first review three weaknesses resulting from the use of QDataStream, which can result in two vulnerabilities. We then select a suitable analysis technique and tool to detect them. Finally, we develop a two-step approach to generate the static analysis checks and to analyze the selected projects from the prevalence analysis.

4.1.1 QDataStream Weaknesses

To identify a vulnerability, we focus on the definitions given in Section 2.1. Two conditions must be met. First, the use of QDataStream must lead to a security failure, meaning an impairment of a security objective, i.e., confidentiality, integrity, or availability. Second, an attacker must be able to trigger this impairment. A vulnerability arises from the combination of a weakness in the code that permits the security failure and an exposure condition that puts the code within an attacker's reach. This section identifies weaknesses introduced by QDataStream and determines which of the two conditions each contributes to. For each weakness we also note the severity in terms of impact and exploitability for the resulting vulnerability.

Research on the weaknesses introduced by the use of QDataStream is sparse, so we base our analysis primarily on the work of Hüther et al. (2023) and Huth (2026). Huth identified multiple risks that can arise from its use, several of which stem from concrete weaknesses, while Hüther et al. focused specifically on the deserialization operator. Two further sources are the QDataStream documentation[1] and implementation[2]. We use all four to get a comprehensive view on the subject, focusing on Qt version 6.11.

[1] `https://doc.qt.io/qt-6/qdatastream.html`, last accessed on 23.07.2026

[2] `https://code.qt.io/cgit/qt/qtbase.git/tree/src/corelib/serialization/qdatastream.cpp?h=6.11`, last accessed on 24.07.2026

Weakness 1: Deserializing Untrusted Data (W1) QDataStream allows for deserialization of data from a QIODevice, which can, for example, be a file or a network connection, potentially provided or controlled by an attacker. Hüther et al. (2023, p. 31) and Huth (2026, p. 34) state that a significant risk associated with QDataStream comes from deserializing untrusted data. CWE-502 covers this case, listing impairment of integrity and availability as possible consequences, depending on how the data is used.[3] W1 itself does not impair integrity or availability, but is the enabling exposure condition that, in combination with the following weaknesses, contributes to a vulnerability. The impact and exploitability depend on the weakness it enables. The severity of the resulting vulnerabilities scales with how reachable the QIODevice is for the attacker, an unauthenticated network connection being more severe than a file the user opens deliberately.

Weakness 2: Reading Raw Binary Data (W2) QDataStream allows reading and writing raw binary data from and to the stream with the functions `readBytes`, `writeBytes`, `readRawData`, and `writeRawData`.[4] We focus on the reading functions `readRawData` and `readBytes` because their input could potentially be attacker-controlled. To better understand the risk, we reviewed their implementations. `readRawData` reads `len` bytes into the preallocated buffer `s`, without checking that the buffer is large enough. A resulting vulnerability has two requirements at the call site: `len` must derive from stream content, and `s` must not be sized from that same value. Where both hold, this results in an out-of-bounds write, described in CWE-787[5], with impairment of integrity and availability as consequences. Listing 4.1 shows the vulnerable part of an example we wrote to demonstrate W2. The function `readRecord`, which is later used with a QDataStream bound to an attacker-provided file, causes a stack buffer overflow when `len` is greater than the buffer. The full example can be found in the thesis repository.[6] The impact is high, while exploitability depends on W1, i.e., whether the stream content reaching `len` is attacker-controlled input and how reachable the input source is.

C++

```
6  void readRecord(QDataStream &in) {
7      char buffer[16];                 // fixed-size destination
8      qint32 len = 0;
9      in >> len;                       // len is attacker-controlled
10     in.readRawData(buffer, len);     // buffer overflow
11     qDebug() << "read" << len << "bytes into a 16-byte buffer";
12 }
```

Listing 4.1: Buffer Overflow Example.

`readBytes` is not exposed in this way: the implementation range-checks the length parameter and allocates incrementally, so a length exceeding the available data fails on the

[3] `https://cwe.mitre.org/data/definitions/502.html`, last accessed on 23.07.2026

[4] `https://doc.qt.io/qt-6/qdatastream.html#reading-and-writing-raw-binary-data`, last accessed on 26.06.2026

[5] `https://cwe.mitre.org/data/definitions/787.html`, last accessed on 23.07.2026

[6] `https://github.com/paul-bode/bachelors-thesis`

first block read rather than being allocated up front. The static analysis therefore targets `readRawData` only.

Weakness 3: Uncontrolled Resource Consumption (W3) QDataStream does not verify that the amount of memory the deserialization operator requests is reasonable.[7] Huth (2026, p. 24) showed that deserializing a large file can lead to high memory usage. This is an example of CWE-400, uncontrolled resource consumption, which may impair availability.[8] Examining the implementations of the deserialization operators for QByteArray and QString showed that memory allocation is not driven by the length prefix alone. The operators allocate incrementally in blocks and abort with `ReadPastEnd` once the stream is exhausted. We tested this by generating a file that announces a large payload which it does not contain, aiming to exhaust memory from a small footprint. Our test showed that a file declaring a large element count but containing no payload aborted without measurable allocation. On the other hand, a file whose elements were fully present allocated approximately 5.7 times its size for a `QByteArrayList` of empty elements, scaling linearly with input size. This means that an attacker should not be able to craft a file of a few bytes in size that leads to allocation of gigabytes of memory. Note that this only holds for an allocation by the `>>` operator; the allocation using the `resize()` function in Section 4.2.3 behaves differently. The program we used to test this can be found in the thesis repository.

QDataStream nevertheless imposes no ceiling of its own, so an attacker being able to deliver a sufficiently large stream can still exhaust memory, and any limit has to be imposed by the developer before the read. Impact on availability is therefore possible, while exploitability of this operator-driven form is limited by the need to supply a payload of comparable size; the `resize()` variant in the case study is not so bounded.

One of these weaknesses alone does not create a vulnerability in a project; the combination of W1 with W2 or W3 does. Our analysis therefore searches for co-occurrences of W1 with W2 or W3, rather than for the individual weaknesses.

Our selection of weaknesses discussed here is bounded by the sources available. Weaknesses arising from user-defined `operator>>` implementations or from operators we did not examine are outside this scope. The set is therefore not exhaustive, and the results in Section 4.2 should be viewed as a lower bound on the number of affected projects.

RQ2: Which QDataStream weaknesses discussed by Hüther et al. (2023) and Huth (2026) could introduce a vulnerability into a Qt project?

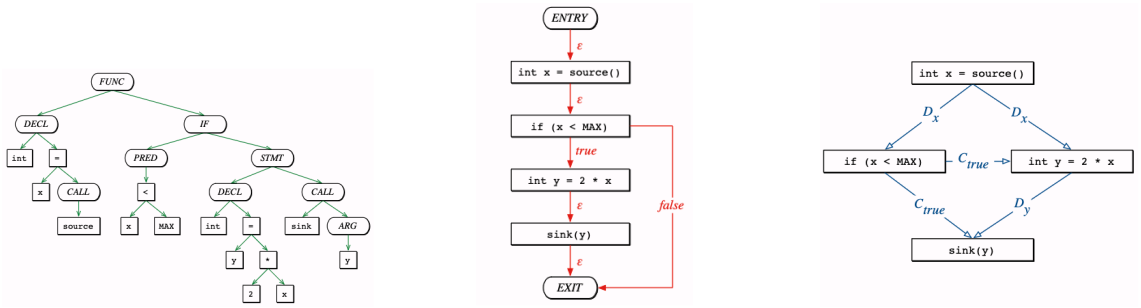
QDataStream itself introduces no vulnerability. It introduces weaknesses that become vulnerabilities in combination: W1+W2 yielding an out-of-bounds write (impairment of integrity and availability) and W1+W3 yielding memory exhaustion (impairment of availability). Finding a combination of W1+W2 or W1+W3 in a project is an indicator of a vulnerability.

[7] `https://doc.qt.io/qt-6/qdatastream.html#corruption-and-security`, last accessed on 26.06.2026
[8] `https://cwe.mitre.org/data/definitions/400.html`, last accessed on 24.07.2026

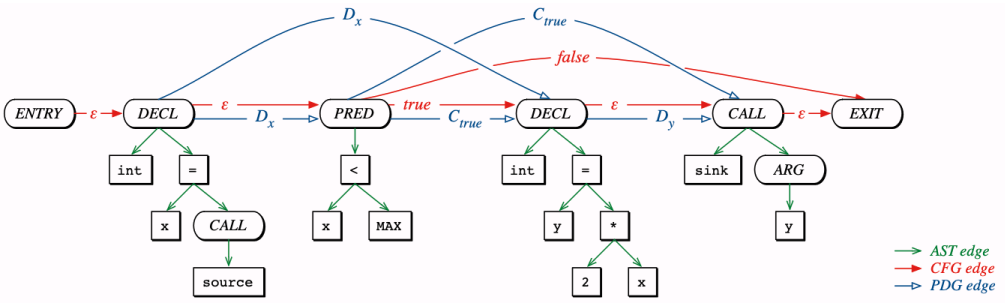
4.1.2 Analyzing Source Code

To answer RQ3, we require a way to identify the weaknesses defined above in the source code of a project. Binkley (2007, p. 2) defines source code analysis as "the process of extracting information about a program from its source code". According to the same author, it consists of three components: "the parser, the internal representation, and the analysis of this representation" (Binkley 2007, p. 3).

The parser translates the source code of the program into an abstract internal representation that will be used for the analysis. The chosen representation depends on the goal of the analysis. Common examples, shown in Figure 4.1, are abstract syntax trees (ASTs), control-flow graphs (CFGs), and program dependence graphs (PDGs), each defined below. ASTs represent the program as a tree (Figure 4.1a) without taking the execution order of statements into account, making them suitable for "flow-insensitive analyses" (Møller et al. 2025, p. 15). CFGs, on the other hand, are directed graphs with the edges representing the control flow (Figure 4.1b), allowing for control-flow-sensitive analysis (Møller et al. 2025, p. 16). PDGs (Figure 4.1c) combine data and control dependencies into one graph, allowing for more efficient algorithms on that data structure and data-flow-sensitive analysis (Ferrante et al. 1987, pp. 320–322). A more complex internal representation is the code property graph (CPG) introduced by Yamaguchi et al. (2014, p. 593). The CPG combines the AST, CFG, and PDG into a unified graph, as shown in Figure 4.1d. This combination of code representations can cover more types of vulnerabilities than any single representation on its own (Yamaguchi et al. 2014, p. 598).



(a) AST (b) CFG (c) PDG



(d) CPG

Figure 4.1: Internal Representations. Taken From Yamaguchi et al. (2014).

Binkley (2007, p. 4) classifies the analysis of the internal representation along six dimensions:

1. static versus dynamic analysis
2. sound versus unsound analysis
3. safe versus unsafe analysis
4. flow-sensitive versus flow-insensitive analysis
5. context-sensitive versus context-insensitive analysis
6. complexity of the analysis

We are only concerned with analysis techniques that depend on the source code and not on the input during program execution. We therefore focus on static analysis and leave dynamic code analysis out of scope.

The terms sound and safe are used inconsistently in the literature, as their precise meaning depends on which property an analysis aims to verify (Møller et al. 2025; Yi et al. 2020; Binkley 2007). To avoid this ambiguity, we describe analyses directly in terms of false positives and false negatives, relative to our goal of detecting the weaknesses defined above. A false positive is a reported occurrence of a weakness that is not actually present, whereas a false negative is a real occurrence that the analysis fails to report. An analysis that avoids all false negatives never misses a weakness but tends to over-report, producing more false positives. On the other hand, an analysis that avoids false positives only reports genuine occurrences but may miss some. Binkley (2007, p. 4) observes this trade-off, noting that the stricter, no-miss analyses tend to produce more false positives, while the more permissive ones can still yield useful results. For our purposes, we aim for a balanced approach that detects most actual weaknesses while accepting a number of false positives.

The weaknesses we want to find impose varying requirements on flow- and context-sensitivity. W1 requires determining whether the data reaching a deserialization originates from an attacker-reachable QIODevice. Because the device is often bound to the stream in a different function than the one performing the read, this requires context-sensitive analysis that takes interprocedural data flow into account. W2 requires both that the length argument of `readRawData` derives from stream content and that the destination buffer is not sized from that same value. The first is a data flow question, while the second is order-dependent: a resize of the buffer between reading the length and using it in `readRawData` would make the call safe. Detecting W2 therefore requires flow-sensitive analysis, and context-sensitive analysis wherever the buffer is allocated outside the function performing the read. W3 is the least demanding, as the deserialization call itself can be found flow-insensitively; establishing that no size check precedes it, however, again requires flow-sensitivity. Flow- and context-sensitive analysis also improves precision and reduces false positives at the cost of computational complexity (Binkley 2007, pp. 4–5). Because of these requirements, we use both flow- and context-sensitive analysis.

4.1.3 Static Analysis Tool Selection

For the selection of the static analysis tool, several constraints apply. The prevalence analysis only included C++ projects, so the tool needs to work with that language. Besides that, we want to easily share the resulting checks with developers, who should be able to use them free of charge, including in commercial contexts.

As a result, we use Joern[9], a tool that implements the CPG described above, to develop checks for all three weaknesses. Joern is open-source and freely usable, including commercially.[10] It allows for source-to-sink analysis and was one of the most widely used tools for detecting software vulnerabilities in 2024 (Shiri Harzevili et al. 2024, p. 24). Joern is a standalone application that generates a queryable CPG for an imported project without requiring the code to compile. Because Joern exposes the CPG through a Scala API, checks can be written in Scala and integrated directly into a continuous integration pipeline or run manually by a developer.

Several other static analysis tools were considered but did not match our constraints. GitHub CodeQL[11], for example, can be used free of charge for research and open-source purposes, but not commercially. While the projects in this analysis are open-source and would allow for analysis with CodeQL, commercial projects could not benefit from the resulting checks. Other popular static analyzers for C++ such as Coverity[12] and SonarQube[13] are proprietary and not freely usable for our purposes. This also applies to Axivion[14], the static code analyzer of the Qt Group itself.

Static analysis tools have limitations. Not all weaknesses, or program properties more generally, can be checked automatically, because some are undecidable (Møller et al. 2025, pp. 6–7). Determining whether an arbitrary program contains a weakness amounts to deciding the `has-weakness` property. Where this property is both non-trivial and semantic, that is, a property of the program's behavior rather than of its syntax, Rice's theorem shows it to be undecidable (Hopcroft et al. 2007, Thm. 9.11). Additionally, static analysis is bound to the available code and cannot reason about the intent of a developer (Hüther et al. 2023, p. 31). Moreover, to the best of our knowledge, no existing static analyzer provides dedicated checks for the QDataStream weaknesses defined above.

Large language models could address the latter two limitations. Used as analyzers themselves, they can take context into account that is unavailable to static analysis. Used as generators, they can produce the checks that no existing static analyzer provides.

4.1.4 Large Language Models for Code Analysis

A promising technology for dealing with security vulnerabilities is artificial intelligence (AI) and in particular the widely used large language models (LLMs). They show increasing capability in understanding and generating code, as well as finding vulnerabilities in it (Jelodar

[9] `https://joern.io/`, last accessed on 29.06.2026
[10] `https://github.com/joernio/joern/blob/master/LICENSE`, last accessed on 13.07.2026
[11] `https://codeql.github.com/`, last accessed on 29.06.2026
[12] `https://www.blackduck.com/static-analysis-tools-sast/coverity.html`, last accessed on 29.06.2026
[13] `https://www.sonarsource.com/products/sonarqube/`, last accessed on 29.06.2026
[14] `https://www.qt.io/quality-assurance/axivion-static-code-analysis`, last accessed on 29.06.2026

et al. 2025, pp. 4–9).

One advantage LLMs have over static analysis is the ability to take context and perceived developer intent into account. An LLM could thus, for example, recognize a security-sensitive situation based on the project it is analyzing. This opens the question: why not use an LLM to analyze the source code of the projects for the weaknesses defined above?

A major disadvantage of LLMs is their computational cost and time consumption. Li et al. (2026, p. 11) found that using LLMs to detect vulnerabilities in a software project can consume hundreds of millions of LLM tokens and take days of running time. Traditional static analysis tools, on the other hand, "incur negligible overhead, completing analysis within seconds to minutes", making them more scalable (Li et al. 2026, p. 11).

Agentic AI is a technology that can potentially balance these advantages and disadvantages. It is a software system based on LLMs that can break down complex problems and solve them using external tools, such as compilers, analysis tools, or the shell, with minimal human intervention (Bandi et al. 2025, pp. 7–9). This allows an agent not only to write static analysis checks but also to execute them and iterate on the results.

Liu et al. (2025, pp. 3–4) proposed AutoChecker, an LLM-based tool that generates checks from a description and accompanying tests, iteratively refining them against those tests. Similarly, KNighter generates static analysis checks from patch commits in an automated fashion (Yang et al. 2025, pp. 2–4). These approaches suggest that an LLM can generate reusable static analysis checks, incurring the LLM cost once rather than on every analysis.

4.1.5 Our Approach

For the vulnerability analysis, we take the two-step approach shown in Figure 4.2. We develop three static analysis checks by using an LLM and run these on the 100 highest-ranked projects from the prevalence analysis conducted in Chapter 3. Modifying the approach of KNighter and AutoChecker, we aim to combine two strengths: the ability of LLMs to write static analysis checks, and the computational efficiency of static analysis.

In Step 1, we generate and validate Joern checks for the three weaknesses. First, we feed the description of the three weaknesses defined above and additional sources into an LLM. Additional sources are links to the QDataStream documentation and implementation, the Quassel CVE discussed earlier, and relevant parts of the work of Hüther et al. and Huth. The LLM analyzes the weaknesses and produces a detailed report describing how to detect them. It also writes positive test cases, which the checks should flag, and negative test cases, which resemble the weakness but should not be flagged. In subsequent executions, the LLM generates static analysis checks for Joern and validates them against the positive and negative test cases written before. The LLM executes Joern to test its own checks against the test cases, iterating on the checks until they pass.

Because comparing different LLMs is outside the scope of this thesis, we rely on an existing LLM coding leaderboard[15] for the model choice. We use Claude Opus 5 because, at the time of the analysis, it was the highest-rated LLM we could reliably access. Claude Opus 5 can be accessed through several interfaces: a chat application (via web, desktop, or

[15] `https://benchlm.ai/`, last accessed on 27.07.2026

mobile), programmatic API access, and agentic tools such as Claude Code that can read and act on a codebase directly. To allow the iterative approach when writing the checks, we use the agentic tool Claude Code.[16] We publish the exact commands and prompts, full session logs, the generated checks, and the resulting reports in the thesis repository.

Since the LLM validates its checks against test cases it wrote itself, the loop in Step 1 shows only that the checks work on tests they were written for. Verification against independent tests is what establishes that they detect the weaknesses as defined. For this verification we use the two examples that we wrote to demonstrate W2 and W3. Both also realize W1, since the deserialization input is read directly from a file, so each example constitutes a complete vulnerability (W1+W2 and W1+W3 respectively). These examples were not available to the LLM during Step 1. This verification therefore establishes that the checks are not merely fitted to the LLM's own test cases, but are able to find a real vulnerability. It does not measure their precision: in both verification cases the code is vulnerable, so false positives remain unquantified. Measuring precision would require cases in which a similar pattern occurs without the vulnerability being present. A check must detect the vulnerability in its corresponding example to proceed to Step 2; if it does not, we manually examine and improve it.

In Step 2, we analyze the 100 highest-ranked projects from the prevalence analysis using the checks from Step 1. To partially address the false-negative limitation of the prevalence analysis, we add Bitcoin Core — which the GitHub search had missed — to the ranking, scoring it the same way as the other projects. It ranks 11th, so every subsequent project moves down one place, and the project previously ranked 100th drops out, keeping the total at 100. For the static analysis, we use a Python script that can be found in the thesis repository. We clone each project from GitHub and record the hash of the analyzed commit for reproducibility. Then we parse the project into a CPG with Joern and run the checks on it. A project failing at clone, CPG construction, or check execution is recorded as an error and excluded from the analyzed set. The coverage gap is reported in the results.

The checks report each weakness they find, along with the file, function, and line in which it occurs. The analysis script additionally searches the findings for co-occurrences of W1+W2 and W1+W3. It then assigns each project to one of the following categories, in decreasing order of criticality:

- W1+W2 and W1+W3
- W1+W2 only
- W1+W3 only
- No co-occurrence

Since W1+W2 might impair integrity in addition to availability, it ranks higher than W1+W3. The number of projects with at least one reported co-occurrence answers RQ3. These co-occurrences also prioritize candidates for the manual inspection: a project with a possibility

[16] `https://claude.com/product/claude-code`, last accessed on 06.07.2026

of both W1+W2 and W1+W3 should be examined first. We conduct such a manual inspection in the form of a case study for the projects from the highest category to determine whether the checks detected an actual vulnerability. The result of this case study answers RQ4.

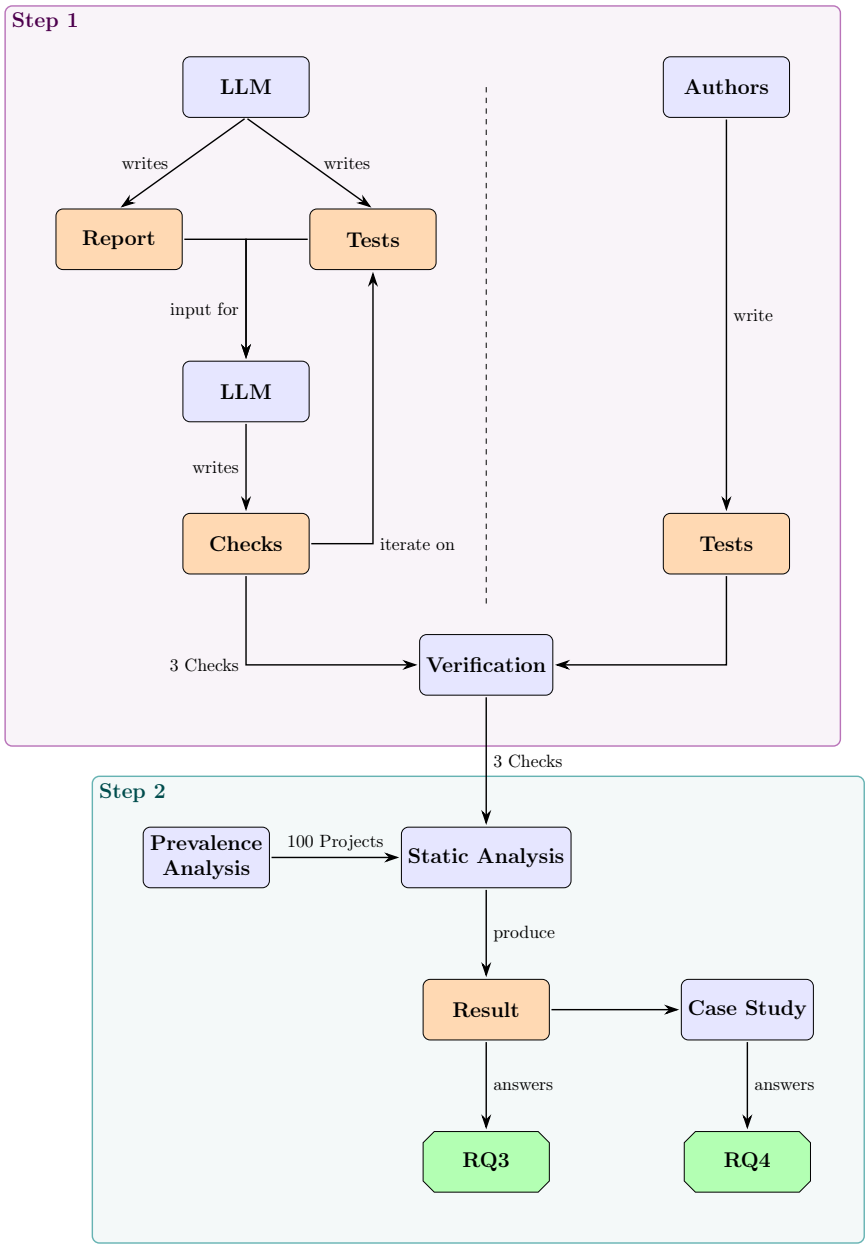


Figure 4.2: Vulnerability Analysis Approach.

4.2 Results

We used Claude Code to implement three Joern checks for the static analysis. The following heavily condensed excerpt from the W1 check illustrates how such a check works. The full source code of the checks can be found in the thesis repository.

The check first identifies all sites where a QDataStream is bound to a device or buffer (Listing 4.2). The bound expression is the potential source. Each bind is associated with a

stream key, which identifies the stream object across method boundaries, so that a bind in one method can be linked to the reads performed on it in another.

```scala
sealed trait Key
case class LocalKey(mid: Long, name: String) extends Key
case class MemberKey(cls: String, name: String) extends Key
case class GlobalKey(name: String) extends Key
case class MethodWide(mid: Long) extends Key
//...
val ctorBinds = cpg.call.nameExact("QDataStream").filter(_.argument.size >= 2).l
val setDevBinds = cpg.call.nameExact("setDevice")
    .filter(c => c.argumentOption(0).exists(a => norm(exprType(a)).matches(StreamType))).l

case class Bind(call: Call, device: Expression, key: Key, writeOnly: Boolean)
//...
val binds: List[Bind] = (ctorBinds.flatMap { c =>
    val m = c.method
    val dev = c.argumentOption(1)
    val writeOnly = c.argumentOption(2).exists(a => a.code.contains("WriteOnly"))
    val key = ctorTarget(c).flatMap(t => keyOf(t, m)).orElse {
      mLocals(m).values
        .find(l => l.typeFullName.matches(StreamType) && !l.name.startsWith("<tmp>") &&
                   l.name != "QDataStream" && l.lineNumber == c.lineNumber)
        .map(l => LocalKey(m.id(), l.name))
    }.orElse(Some(MethodWide(m.id())))
    dev.map(d => Bind(c, d, key.get, writeOnly))
} //...
```

Listing 4.2: W1 Check: Identifying QDataStream Bound Sites.

The sources are classified into the categories `Untrusted`, `Internal`, and `Unknown` (Listing 4.3). A source such as a TCP socket falls into the category `Untrusted`, while a temporary file provided by the application itself is `Internal`.

```scala
sealed trait Verdict
case object Untrusted extends Verdict   // crosses a trust boundary
case object Internal  extends Verdict   // tier E, provably program controlled
case object Unknown   extends Verdict   // no evidence either way
//...
def classify(e: Expression, m: Method, depth: Int = 0): Verdict = {
    //...
    if (ty.matches(UntrustedDeviceType)) return Untrusted
    if (ty.matches(TempFileType)) return Internal
    //...
}
```

Listing 4.3: W1 Check: Classifying Sources.

Sources that remain `Unknown` after this, such as a device obtained from an opaque call, are passed to a narrow taint query (Listing 4.4). It runs `reachableBy` — which returns the source nodes from which data flows to the bound device — only for these few binds, avoiding a global data-flow analysis. The resulting verdicts determine which stream keys are untrusted.

Sinks are identified by the stream's read operations (Listing 4.5).

```scala
val taintUntrusted: Set[Long] = {
    val ambiguous = readBinds.filter(b => classify(b.device, b.call.method) == Unknown)
    if (ambiguous.isEmpty) Set.empty[Long]
    else {
      val extSources = cpg.call.filter(isExternalSourceCall).l
      ambiguous.filter { b =>
        val m = b.call.method
        val localSources: List[CfgNode] =
          m.parameter.filter(_.name != "this").l ++
            mCalls(m).filter(c => norm(exprType(c)).matches(UntrustedDeviceType))
        try Iterator(b.device).reachableBy((extSources ++ localSources).iterator).nonEmpty
        catch { case _: Throwable => false }
      }.map(_.call.id()).toSet
    }
}

def computeUntrustedKeys(): Set[Key] = readBinds.collect {
    case b if classify(b.device, b.call.method) == Untrusted => b.key
    case b if taintUntrusted.contains(b.call.id()) => b.key
  }.toSet
```

Listing 4.4: W1 Check: Taint Query on Sources Classified Unknown.

```scala
val shiftReads = cpg.call.nameExact("<operator>.arithmeticShiftRight").l
val fnReads = cpg.call.nameExact("readRawData", "readBytes", "skipRawData", "operator
>>").l
val reads: List[Call] = (shiftReads ++ fnReads).filter { r =>
    streamExprOf(r).exists(e => isStreamExpr(e, r.method))
}
```

Listing 4.5: W1 Check: Identifying Sinks.

With sources and sinks in place, the check identifies the connections between them to find untrusted reads (Listing 4.6). A value read from an untrusted stream is itself untrusted, so a stream constructed over it becomes untrusted in turn; the loop therefore iterates until the set of untrusted reads stops growing. `untrustedReads` contains the candidate findings.

Using `isSanitized`, a candidate is discarded if a call whose name suggests an integrity check happens before the read. This is a name-based heuristic: it establishes that such a call is executed beforehand, not that the verification is sound or that its result is acted upon. Each surviving read is emitted as `FINDING|W1|<file>|<function>|<line>`.

Scala

```
var untrustedKeys: Set[Key] = Set.empty
var untrustedMethods: Set[Long] = Set.empty
var untrustedReads: List[Call] = Nil
var grew = true
var round = 0
while (grew && round < 8) {
    round += 1
    grew = false
    untrustedKeys = computeUntrustedKeys()
    //...
    untrustedMethods = untrustedKeys.collect { case MethodWide(mid) => mid }
    val boundReads = reads.filter { r =>
        streamExprOf(r).flatMap(e => keyOf(e, r.method)).exists(untrustedKeys.contains) ||
        untrustedMethods.contains(r.method.id())
    }
    val helperReads = helperStreamReads.flatMap { case (rds, argKeys) =>
        val anyUntrusted  = argKeys.exists(untrustedKeys.contains)
        val anyClassified = argKeys.exists(allBindKeys.contains)
        if (anyUntrusted || !anyClassified) rds else Nil
    }
    untrustedReads = (boundReads ++ helperReads).distinct
    untrustedReads.foreach { r =>
        destVarName(r).foreach(n => if (streamDerived.add((r.method.id(), n))) grew =
true)
    }
    //...
}
//...
def isSanitized(r: Call): Boolean = {
    val m = r.method
    val guards = mCalls(m).filter(c => c.name.matches(Sanitizer))
    if (guards.isEmpty) false
    else sanitizedCache.getOrElseUpdate(r.id(), {
      val rl = r.lineNumber.map(_.toInt).getOrElse(Int.MaxValue)
      guards.exists { g =>
        val gl = g.lineNumber.map(_.toInt).getOrElse(Int.MaxValue)
        gl <= rl && (try g.dominates.id.toSet.contains(r.id()) catch { case _: Throwable
=> true })
      }
    })
  }
```

Listing 4.6: W1 Check: Finding Untrusted Reads.

4.2.1 Check Verification

To confirm that the checks identify the discussed vulnerabilities, we tested them on our two example cases. Table 4.1 shows the verification results: each check reported its weakness in the test case that contained it. No check reported a weakness in a case not containing it. All three checks therefore passed verification and required no changes to enter Step 2. The checks along with the verification report can be found in the thesis repository.

	Test case	
	W2 example	W3 example
Weaknesses present	W1, W2	W1, W3
Check W1	✓	✓
Check W2	✓	–
Check W3	–	✓

Table 4.1: Verification results. ✓ denotes that the check reported the right weakness in this case.

4.2.2 Static Analysis

Running the three checks written by the LLM on the 100 highest-ranked projects from the prevalence analysis resulted in an initial error rate of 26 %. For 26 out of the 100 projects, the analysis did not complete for one of two reasons. In three cases, the Joern parser timed out while creating the CPG. We did not investigate the individual causes; possible explanations include memory constraints on our hardware and Joern's difficulties with parsing specific files. In 23 cases, check W3 raised a `NullPointerException`: the check dereferenced the enclosing method of a CFG node, but the node had no enclosing method in the CPG, so the lookup returned `null`. The LLM did not take this into account and failed to perform proper error handling. Before running the analysis again, we manually fixed W3 and also reviewed the other two checks for obvious errors.

The overall results of the analysis are shown in Table 4.2. Of the 100 attempted projects, the same three as before failed because of a timeout while creating the CPG. Of the 97 projects that were analyzed successfully, 75 (77.3 %) had indications of at least one of the weaknesses. In 22 projects (22.7 %), none of the three checks reported a finding. As defined earlier, a vulnerability requires the combination W1+W2 or W1+W3; this was indicated for 51 projects (52.6 %): 47 with W1+W3 only, none with W1+W2 only, and four with both W1+W2 and W1+W3 (4.1 %). Every project that triggered W2 also triggered W3, so the number projects with at least one finding equals the W3 count. The full report, with findings linked to files, function names, and line numbers, can be found in the thesis repository. In summary, the results of this analysis allow us to answer the third research question.

RQ3: How many of the 100 highest-ranked projects from the prevalence analysis show indications of vulnerabilities introduced by use of QDataStream?

Our checks indicated at least one of the weaknesses in 75 out of the 97 successfully analyzed projects (77.3 %). In 51 out of the 97 successfully analyzed projects (52.6 %), we found a combination of at least two weaknesses that indicates a vulnerability.

	Projects	Share
Coverage		
Attempted	100	
Failed to analyze	3	3.0 %
Successfully analyzed (n)	97	
Individual weaknesses		
W1 (deserializing untrusted data)	75	77.3 %
W2 (raw binary read)	4	4.1 %
W3 (uncontrolled resource consumption)	51	52.6 %
Co-occurrence categories		
W1+W2 and W1+W3	4	4.1 %
W1+W2 only	0	0.0 %
W1+W3 only	47	48.5 %
No co-occurrence	46	47.4 %
Projects with findings		
At least one finding	75	77.3 %
No findings	22	22.7 %

Table 4.2: Results of the static analysis. The percentages refer to the 97 projects that were successfully analyzed, except for the failed-to-analyze row, which is relative to the 100 attempted projects.

4.2.3 Case Study

The results of the static analysis show indicators of a vulnerability that need to be manually examined to confirm it. Four projects from the static analysis fall into our highest category of indicators, showing potential for both W1+W2 and W1+W3. In all four of them, both W1+W2 and W1+W3 flagged the same function `receiveConnection` in files called `qtlocalpeer.cpp` in three projects and `ttklocalpeer.cpp` in one project. All of them are based on `qtlocalpeer.cpp`, which is part of QtSingleApplication, a component provided by Qt itself.[17] This component can be used to implement an application that can only be started once per user and is thus included in many projects. Searching the remaining results of the analysis for `receiveConnection` revealed six further projects — ten in total — using this code. The relevant lines of `receiveConnection` are shown in Listing 4.7.

[17] `https://github.com/qtproject/qt-solutions/tree/master/qtsingleapplication`, last accessed on 01.08.2026

```C++
// socket = QLocalSocket* socket = server->nextPendingConnection();
QDataStream ds(socket);
QByteArray uMsg;
quint32 remaining;
ds >> remaining;
uMsg.resize(remaining);
int got = 0;
char* uMsgBuf = uMsg.data();
do {
    got = ds.readRawData(uMsgBuf, remaining);
    remaining -= got;
    uMsgBuf += got;
} while (remaining && got >= 0 && socket->waitForReadyRead(2000));
```

Listing 4.7: QtSingleApplication – qtlocalpeer.cpp.

W1+W2

For W2 to take effect, we identified two requirements: the length argument must derive from the stream, and the buffer must not be sized from that same value. The first requirement is satisfied, as `remaining` is used in `ds.readRawData` without input validation. However, the QByteArray `uMsg` is resized with the same value, preventing a buffer overflow and making this finding a false positive. Interestingly, of the ten projects in our sample that contain `receiveConnection`, four reported W2, and six did not. This is likely due to the surrounding code not being byte-identical and using different versions of the QtSingleApplication component.

W1+W3

QDataStream `ds` is created from a QLocalSocket, accepting connections from a local process. The incoming data from the socket is deserialized into a 32-bit integer value that is then used to resize a QByteArray, without input validation. In theory this should allow one to write a length value of four bytes into the socket, resulting in a memory allocation of around four gigabytes.

Exploiting W1+W3

To test this theory, we wrote a simple Python exploit that auto-detects the socket, connects to it, and sends four bytes. An excerpt of the code is shown in Listing 4.8.

```python
def send(name, declared_len):
    s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
    sock = find_socket(name)
    if sock == None:
        return 0
    try:
        s.connect(sock)
    except OSError as e:
        print("connect failed:", e)
    s.sendall(struct.pack(">I", declared_len))
    s.close()

# Krita W3: crash from 4 bytes
send("krita", 0x7FFFFFFF)

# Tiled W3: 4 GiB resize from 4 bytes
send("tiled", 0xFFFFFFFF)
```

Listing 4.8: W3 Exploit for Krita and Tiled.

We tested the exploit on two projects using the `qtlocalpeer.cpp` component, namely Krita[18] (version 5.3.3) and Tiled[19] (version 1.12.2). We chose these two projects because they had the highest prevalence-analysis scores among the four projects in the highest category of indicators. Additionally, choosing both projects allows us to observe the vulnerability's behavior across different Qt versions. This is important because the four bytes to send, and the expected behavior, depend on the Qt version the targeted application is using. The reason for this lies in how `QByteArray::resize()` is implemented in different versions of Qt. We verified this by comparing the implementation of `resize()` between Qt 5.15.17[20] used by Krita and Qt 6.10.2[21] used by Tiled.

In Qt 5.15.17 `resize()` gets a size value of type `int`, limiting the size to just under two gigabytes. Sending the size parameter `0x7FFFFFFF` results in Krita crashing with a `std::bad_alloc` exception.

In Qt 6.10.2 `resize()` gets a size value of type `qsizetype`, whose size is 8 bytes on the 64-bit platform we tested the exploit on. Sending the size parameter `0xFFFFFFFF` results in Tiled resizing to more than four gigabytes and the program freezing.

In addition to the full code for this first exploit, the thesis repository contains a second exploit for Tiled. It does the same as the first exploit, but at short intervals and automatically reconnects if the application is restarted. This makes the application unusable as long as the script runs. Both exploits were developed and tested on macOS.

Note that the weakness induced by `resize()` does not exactly match the definition of W3, because the `>>` operator is not the vulnerable part, i.e., it does not perform the oversized allocation. Nevertheless, we classify the pattern `x >> length; resize(length)`, where `x` is

[18] `https://github.com/KDE/krita`, last accessed on 01.08.2026

[19] `https://github.com/mapeditor/tiled`, last accessed on 01.08.2026

[20] `https://code.qt.io/cgit/qt/qtbase.git/tree/src/corelib/text/qbytearray.cpp?h=5.15.17#n1787`, last accessed on 03.08.2026

[21] `https://code.qt.io/cgit/qt/qtbase.git/tree/src/corelib/text/qbytearray.cpp?h=6.10.2#n1910`, last accessed on 03.08.2026

attacker-controlled, as a variant of the vulnerability W1+W3. In this case the W3 check flags a stream-read whose value is later used to allocate memory without input validation, which is what let the check find this vulnerability.

Case Study Result

The W1+W3 finding clearly impairs availability and, when triggered from an attacker-controlled local process, warrants classification as a vulnerability. Exploitability, however, is limited, because an attacker needs a compromised process on the machine. The exact consequence — uncontrolled resource consumption or a program crash — depends on the version of Qt used by the target.

We used a CVSS calculator to get an idea of a rough score for this vulnerability. The attack vector is local, and low (non-administrative) privileges are required, because the attacker needs a compromised process with the permissions of the targeted user. Attack complexity is low — a simple Python script suffices — and no user interaction is needed. The attack has no special requirements: it affects both the Qt 5 (int) and Qt 6 (qsizetype) builds we tested, differing only in the resulting failure mode, so the payload and outcome are version-dependent even though exploitability is not. The only impact is on availability — an outright crash in the case of Krita and a freeze in the case of Tiled, which the second exploit sustains for as long as it runs. Only the target system is impaired, not subsequent systems. Using this as input for the calculator results in a CVSS of 6.8 (medium).[22]

Note that an attacker who has compromised a local process — in the case of Krita — has other ways to achieve the same result. For example, by sending an operating system signal that terminates the application. The finding is nevertheless relevant: it stems from a missing input validation in the application, not from the privileges the attacker already holds. The same pattern would be far more severe if the QDataStream were bound to a network source rather than to a local socket. Additionally, the Tiled case shows an impact that signal-based termination does not reproduce, since the process remains alive while consuming resources.

RQ4: Does an indication reported by the analysis correspond to an actual vulnerability in real Qt projects?

We found a vulnerability impairing availability through uncontrolled resource consumption reachable by a local attacker, with a CVSS of approximately 6.8. The vulnerable code is part of a component provided by Qt itself and used in multiple projects, which broadens the vulnerability's reach. We also found that the W2 check produced a false positive.

[22] `https://www.first.org/cvss/calculator/4.0#CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`, last accessed on 03.08.2026

4.3 Vulnerability Analysis Discussion

The approach of our vulnerability analysis is subject to a number of limitations. We identified three weaknesses that could lead to two vulnerabilities. We do not claim that this set is exhaustive: using QDataStream may introduce further weaknesses, and consequently further vulnerabilities, that we did not cover. Identifying them is left to future research.

Using an LLM to work with these weaknesses also has several constraints. Running the same prompt multiple times will most likely result in a considerably different check. This makes the results of check generation not reproducible. Additionally, one of the checks failed in a significant number of cases, even though the LLM tested it on multiple test cases. Further research taking a similar approach should take this into account, e.g., by intentionally giving the LLM incomplete CPGs to prompt proper error handling.

Most importantly, we did not verify the quality of the resulting checks systematically. The verification test and the case study we used only showed that the checks can find a vulnerability, not that they find all vulnerabilities. Two test cases are not enough to ensure the quality of the checks. A systematic evaluation would require a labeled dataset containing both vulnerable and safe uses of QDataStream. That the checks can also produce false positives was demonstrated by the case study; we did not examine to what extent this happens in general. We also saw that the W2 check behaved inconsistently on `receiveConnection`, reporting a weakness in only some cases. This needs to be further investigated.

Joern introduces its own set of limitations to the analysis. We parsed the CPGs directly from the source code of the projects. This means the parser does not have access to the Qt header files, resulting in lower-quality CPGs with missing information. A way to circumvent this would be including the headers, but manually inserting the paths makes the whole setup more fragile and harder for others to adapt and reproduce. Using a compilation database of the project generated by a build tool could solve this, but introduces the need to configure and build the project.

Another limitation when using Joern is posed by hardware constraints. Joern requires a considerable amount of memory for creating the CPG[23], an amount that is not always available on the consumer hardware we used for the analysis. This could be the reason for the timeouts we saw in Section 4.2.2.

A limitation to the actual usability of the checks is their format. The generated checks are `.sc` files that can be shared with and executed by developers, but this format cannot be contributed to Joern directly. Joern has a scanning tool called Joern Scan that takes checks from the Joern Query Database and scans a given project for them. Checks contributed to Joern need to be in this query format. We chose not to use this format because iterating and testing the checks would have required the LLM to rebuild the database each time, making it not feasible for our approach. Although it is possible to translate the queries into the right format, we decided that the checks needed to be tested more thoroughly before they can be contributed. One of the reasons for this is the false positive of the W2 check shown in the case study.

[23] `https://docs.joern.io/installation/#configuring-the-jvm-for-handling-large-codebases`, last accessed on 31.07.2026

Aside from that, the checks explicitly target C++20, Joern 4, and Qt 6. We cannot definitively say how transferable they are to other versions of C++, Joern, or Qt. Given the projects analyzed differ in their C++ and Qt versions, we assume that the checks at least remain applicable for different versions of C++ and Qt. Because we used Joern 4 throughout, no such claim can be made about different Joern versions.

Apart from the LLM and Joern, there are limitations to the analysis in general. When a project shows an indication of one of the weaknesses, it is the job of the developer to interpret this finding and understand the bug or vulnerability at hand. The checks only indicate there is a problem, but do not explain it. Finally, our checks can produce false negatives; there is no guarantee that a project that showed no indication in our analysis is free of these vulnerabilities. As noted in Section 4.1.3, this is to some extent a limitation of code analysis in general.

Nevertheless, we were able to present a workflow from defining the weaknesses and vulnerabilities all the way to detecting them, answering research questions RQ2 through RQ4. The LLM produced three usable Joern checks, only one of which needed manual correction. These checks are a good starting point for future work and can be manually improved or iterated on using real-world results like the false positive shown in the case study. Future improvements of the checks should also differentiate between the different types of a weakness found, e.g., W1 with a TCP socket or W1 with a file. This would make prioritizing vulnerabilities to manually evaluate easier. The case study did show a vulnerability with limited exploitability in at least two projects. In addition, eight other projects using QtSingleApplication could be affected. In line with the goal of this chapter, we defined potential vulnerabilities that using QDataStream could introduce into open-source Qt projects, determined how many projects show indications of them, and verified for two projects that such an indication indeed led to an actual vulnerability.

5. Discussion

The goal of this thesis was the analysis of the impact of QDataStream security risks on open-source Qt projects. We approached this by splitting the risk into two parts, prevalence and vulnerability, which the following section discusses in turn.

5.1 Interpretation of the Results

The prevalence analysis is a quantitative empirical study on how many projects could be affected by the QDataStream security risks. We used a large sample of 69,763 verified Qt projects to identify 7,221 projects that used QDataStream. Because we measured only open-source projects on GitHub, the true number of affected C++ projects is likely higher. The scale of potential exposure makes further analysis worthwhile.

The vulnerability analysis takes a quantitative approach first, identifying vulnerable projects to quantify the risk. To show the risk is not merely theoretical, we confirmed one of the vulnerabilities in a case study. Combining these, we showed a general path from a risk identified in a Qt component to a vulnerability in an open-source project. QDataStream is only one of several Qt risks identified in the report by Hüther et al. (2023). Future research could adapt our approach to get a comprehensive view on not just one Qt class but on the vulnerabilities induced by Qt in general.

In his thesis, Huth (2026) used a small self-contained example to demonstrate possible uncontrolled resource consumption resulting from the use of QDataStream. The author states that bigger examples, such as the ones we used, would allow for analysis of more complex errors in applications (Huth 2026, p. 31). We showed that this is indeed the case.

Hüther et al. (2023, p. 6) conducted three expert interviews with Qt employees, out of whom two talked about QDataStream. One of the concerns of interviewee I was that developers might be using QDataStream to deserialize data from unknown sources (Hüther et al. 2023, p. 6). Our analysis showed that 77.3% of analyzed projects had indications for this. Extrapolated to the 7,221 projects identified by the prevalence analysis, this could be the case for 5,581 projects. Whether this transfer is possible is open for debate, as noted in Section 5.2.

Interviewee I also mentioned that recent changes in the types Qt uses could lead to "confusion and potential numeric overflows" (Hüther et al. 2023, p. 6). While it did not lead to a numeric overflow in our case, we observed this change of types from Qt 5 to Qt 6 in the differing impact of our exploit.

Interviewee II states that QDataStream "was not intended to be used with untrusted data" (Hüther et al. 2023, p. 7). This is consistent with the documentation, which recommends not using QDataStream when the origin of the data is untrusted.[1] That 77.3% of analyzed projects showed indications of exactly this, together with a vulnerability we found in a component provided by Qt itself, prompts the question: Are developers misusing QDataStream systematically?

[1] `https://doc.qt.io/qt-6/qdatastream.html#corruption-and-security`, last accessed on 07.08.2026

The solution to using QDataStream safely in its current state brings us back to Huth. The recommendations from his thesis boil down to treating deserializing as a trust boundary and applying proper input validation (Huth 2026, p. 34). We agree with this: Creating vulnerabilities when using QDataStream is avoidable, and developers should always validate input received from deserialization. Additionally, they should avoid the function `readRawData` entirely.

But what if developers do not use proper input validation on a large scale? Should the implementation of QDataStream be changed to face this reality? Future research should answer this question and make suggestions for a safer version of QDataStream. Hardening the underlying implementation in a way that would take the burden of input validation off developers or explicitly force them to use it, could potentially reduce vulnerabilities in thousands of open-source projects.

5.2 Threats to Validity and Limitations

This thesis can be considered empirical software engineering and thus subject to validity concerns. While designing the methodologies for both analyses, we had to make choices in focus, tools, and techniques. Each of these choices involved trade-offs, which give rise to the limitations of this work. We list the threats to validity and the limitations for each of the four research questions separately. Some limitations that we established in the earlier chapters are repeated here for completeness.

RQ1

The quantitative approach of the prevalence analysis contained multiple steps, each subject to its own threats.

The data selection introduced two limitations. One is inherent to the scope being open-source projects only. Closed-source projects, which account for a difficult-to-determine portion of all projects, are left out. Therefore, no conclusions can be drawn regarding the applicability of the findings to all Qt projects. The second is that we used a single hosting platform for mining the software repositories. Although GitHub is arguably the biggest hosting platform for open-source projects, this leaves an unquantified number of projects unaccounted for. Additionally, the way we identified Qt projects on GitHub does not guarantee that we found all Qt projects hosted there. The example of Bitcoin Core illustrates this.

As mentioned in Chapter 3, the normalization is slightly biased towards younger projects. While the ranking metric based on a popularity heuristic works, its resulting score seems not entirely adequate to prioritize critical projects for a vulnerability analysis.

Finally, the results for the class QDataStream cannot be applied to other Qt classes.

RQ2

The weaknesses and vulnerabilities identified to answer RQ2 are limited by the existing research on the topic. Our result also depends on our chosen definitions of what classifies as a weakness and vulnerability. Depending on where one decides to draw the line, a bug that

corrupts data without an attacker actively exploiting it could be classified as a vulnerability. Conversely, a finding we classify as a vulnerability could be regarded as a bug under a stricter definition. We also did not examine the implementation of all classes implementing the QDataStream deserialization operator and did not cover user-defined versions of it.

RQ3

The results of RQ3 are limited by the checks, Joern, and the coverage of the static analysis itself.

The checks were written by an LLM, and although we reviewed them by hand and fixed an error in the check for W3, they could contain additional errors. Both false negatives and false positives resulting from the checks, and the Joern tool itself, are possible. We gave an example of a false positive in the case study.

Joern failed to parse the CPG for three projects. Configuring it differently, or using hardware with more memory, could solve this issue.

We conducted the static analysis on just the 100 highest-ranked out of the 7,221 QDataStream projects. Because this is not a random sample, our findings might not be applicable to the whole population of projects. Considering the 7,221 QDataStream projects we identified — and the far larger population these may represent — a sample of 100 is very small. A future iteration of the analysis should run the checks on all 7,221 projects.

RQ4

The case study examines two projects of specific versions and on macOS. The results do not allow us to determine whether the vulnerability can also be exploited — in the same way and with the same impact — in other versions of the applications or on other operating systems. Although the code of the function `receiveConnection` for the QtSingleApplication identified in ten projects is identical, we did not verify if each of the projects is indeed vulnerable.

Finally, we classified our finding as a vulnerability according to the definition established in Section 2.1 and disclosed it as such to the affected projects. However, the maintainers of these projects may apply a stricter definition and regard the finding as a bug rather than a vulnerability in its own right, since exploitation presupposes that an attacker has already compromised a local process.

6. Conclusion

To conclude this thesis, we first answer all four research questions before we come to an overall conclusion.

6.1 Answers to the Research Questions

How many open-source projects that use the Qt framework and are written in C++ make use of the class QDataStream? (RQ1)

We identified a sample of 69,763 verified Qt projects, of which 7,221 used QDataStream. The QDataStream class thus has a prevalence rate of $\approx 10.3508\%$. Additionally, older projects are more likely to use QDataStream than younger projects.

Which QDataStream weaknesses discussed by Hüther et al. (2023) and Huth (2026) could introduce a vulnerability into a Qt project? (RQ2)

QDataStream itself introduces no vulnerability; it introduces weaknesses that become vulnerabilities in combination. The first weakness (W1) lies in the deserialization of untrusted data. The second weakness (W2) is using `readRawData` with a length parameter that exceeds the size of the buffer it reads into. An occurrence of W1, supplying an attacker-controlled length parameter to W2, can lead to an out-of-bounds write, impairing integrity and availability. The third weakness (W3) arises from unreasonable memory allocation when deserializing into a class such as QString or QByteArrayList, or when resizing a buffer from QDataStream content, without validating the given size parameter. Combining W3 with an attacker-supplied length prefix can result in uncontrolled resource consumption, impairing availability.

How many of the 100 highest-ranked projects from the prevalence analysis show indications of vulnerabilities introduced by use of QDataStream? (RQ3)

Of the 100 highest-ranked projects from the prevalence analysis, 97 were successfully analyzed. For 75 out of the 97, our checks indicated at least one of the three weaknesses (77.3 %). In 51 out of the 97 successfully analyzed projects (52.6 %), we found a combination of at least two weaknesses that indicates a vulnerability.

Does an indication reported by the analysis correspond to an actual vulnerability in real Qt projects? (RQ4)

In a case study we examined two projects and found a vulnerability impairing availability through uncontrolled resource consumption. The vulnerability is exploitable by a local attacker without administrative privileges, which we assessed at a CVSS of approximately 6.8. The vulnerable code is part of the QtSingleApplication component provided by Qt itself and used in at least ten of the analyzed projects, which broadens the vulnerability's reach. We also found that the check for W2 produced a false positive.

6.2 Overall Conclusion

In this thesis we showed that QDataStream is used in about one in ten Qt open-source projects hosted on GitHub. Of the two vulnerabilities we identified, we confirmed one in at least two real projects. These projects, Krita and Tiled, are actively used and both have over ten thousand stars on GitHub.

Although the exploitability of this particular vulnerability we discovered is limited, and large-scale exploitation is unlikely, it still demonstrates the need for further research. The checks we provide should be understood as a starting point for that. They are freely available in the thesis repository, allowing for use by developers and researchers alike.

Beyond QDataStream, Hüther et al. (2023) enumerate a long list of potentially vulnerable Qt classes that should be analyzed similarly. This would allow for a better understanding of the prevalence and vulnerability of Qt in general. For this, the prevalence analysis pipeline and check-generation methodology could be employed after further improving them.

7. Ethics Considerations

In order to comply with the USENIX Security open science policy[1], we share all research artifacts associated with this thesis publicly, with the temporary exception described below. All source code, scripts, checks, prompts, and reports can be found in the thesis repository.[2] Because of the security-sensitive nature of our findings, we need to address the ethical implications of our work. Our considerations are based on the three ethical guidelines and policies of the conferences USENIX Security, IEEE Symposium on Security and Privacy[3], and Network and Distributed System Security Symposium[4]. This thesis produced four main artifacts that could harm individuals or organizations.

We created a pipeline of four Python scripts that are capable of providing a list of Qt projects that contain a specific keyword. This keyword could be a Qt class or function name, a vulnerable line of code, or a specific value — something an attacker could abuse. There are several existing tools capable of this sort of scanning, e.g., `grep.app`[5]. We also provide the full lists of projects found to be using QDataStream. The list is not harmful in itself, but combined with the third artifact, the static analysis checks we provide, an attacker could identify vulnerabilities in these projects. From our point of view, the potential harm of these artifacts is outweighed by their value for future research on specific Qt classes.

For the case study in the vulnerability analysis, we wrote the fourth artifact, Python code that exploits the vulnerability in Krita and Tiled, and that could potentially affect at least ten projects sharing the same component. We decided to publish the code because it illustrates the vulnerability while the impact and exploitability are quite limited. An attacker needs to be able to start a local process, and even then can only impair availability locally. Once an attacker has this level of access, the same impact on availability could be achieved through other means, such as terminating the process directly with an operating system signal. Because of this, we consider active exploitation in the wild, using our exploit code, to be highly unlikely. Additionally, the projects we examined in the case study are not safety-critical: exploitation does not endanger human health and is unlikely to cause significant financial loss.

We disclosed the vulnerability to three parties: to Krita and Tiled, so they can fix the problem, and to Qt, so it can provide a fix that benefits all projects using QtSingleApplication. We opted for responsible disclosure, giving the three projects at least 90 days to address the vulnerability before we make it public and publish the exploit. This 90-day period is a default: we will extend it if the affected projects require more time, and may publish earlier if fixes are released and the maintainers agree. At the time of writing, the disclosure process is still ongoing. Because of this and the deadline for this thesis, we decided on the following approach. At the time the bachelor's thesis is submitted and evaluated — which falls within the initial 90-day period — only the thesis's two supervisors will have access to the artifacts

[1] `https://secartifacts.github.io/usenixsec2026/ethics-open-science`, last accessed on 08.08.2026
[2] `https://github.com/paul-bode/bachelors-thesis`
[3] `https://sp2027.ieee-security.org/cfpapers.html`, last accessed on 08.08.2026
[4] `https://www.ndss-symposium.org/ndss2027/submissions/call-for-papers/`, last accessed on 08.08.2026
[5] `https://grep.app/`, last accessed on 08.08.2026

in a private university repository. After the disclosure period has elapsed, the artifacts will be published in a public GitHub repository, along with an updated version of this thesis reflecting the change from the private GitLab repository to a public release on GitHub. This lets us balance responsible disclosure with the USENIX Security open science policy, ultimately publishing our artifacts for future research.

Use of AI-Based Applications

We used several AI applications in the preparation of this bachelor's thesis. Our use of AI followed the *Guidelines for the Use of AI-Based Systems at the University of Bremen*[1], in the version adopted by the University Executive Board on 30.06.2026. In addition to the use of AI described in the vulnerability analysis methodology (Section 4.1.5), we used the following AI applications.

Claude We used the Claude[2] large language models Fable 5, Opus 5, and Opus 4.8 to understand, write, and debug Python, C++, and Scala code. We also used them for literature search and proofreading. For the literature search, we used Claude solely to identify candidate references. Every work cited in this thesis was retrieved in the original, and all statements attributed to it were verified against that original. For proofreading, we used Claude to check spelling, grammar, and style. It also suggested alternative formulations, which we reviewed and adapted.

DeepL We used the DeepL[3] translator to translate words and sentences between German and English.

[1] `https://www.uni-bremen.de/fileadmin/user_upload/sites/digitrans/Checklisten/Guidelines_for_the_Use_of_AI_Based_Systems_v2-0-2.pdf`, last accessed on 08.08.2026

[2] `https://claude.com/product/overview`, last accessed on 08.08.2026

[3] `https://www.deepl.com/en/translator`, last accessed on 08.08.2026

Bibliography

Anderson, Ross (2020). *Security Engineering: A Guide to Building Dependable Distributed Systems*. Third edition. Indianapolis: John Wiley & Sons. ISBN: 978-1-119-64278-7 978-1-119-64468-2 978-1-119-64283-1 978-1-119-64281-7.

Bandi, Ajay, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala (Sept. 2025). "The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges". In: *Future Internet* 17.9, p. 404. ISSN: 1999-5903. DOI: 10.3390/fi17090404. (Visited on 07/13/2026).

Binkley, David (May 2007). "Source Code Analysis: A Road Map". In: *Future of Software Engineering (FOSE '07)*, pp. 104–119. DOI: 10.1109/FOSE.2007.27. (Visited on 06/19/2026).

Borges, Hudson, Andre Hora, and Marco Tulio Valente (Oct. 2016). "Understanding the Factors That Impact the Popularity of GitHub Repositories". In: *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 334–344. DOI: 10.1109/ICSME.2016.31. (Visited on 06/19/2026).

Borges, Hudson and Marco Tulio Valente (Dec. 2018). "What's in a GitHub Star? Understanding Repository Starring Practices in a Social Coding Platform". In: *Journal of Systems and Software* 146, pp. 112–129. ISSN: 0164-1212. DOI: 10.1016/j.jss.2018.09.016. (Visited on 06/01/2026).

Ferrante, Jeanne, Karl J. Ottenstein, and Joe D. Warren (July 1987). "The Program Dependence Graph and Its Use in Optimization". In: *ACM Transactions on Programming Languages and Systems (TOPLAS)* 9.3, pp. 319–349. ISSN: 0164-0925. DOI: 10.1145/24039.24041. (Visited on 07/06/2026).

Grochowski, Konrad, Michał Breiter, and Robert Nowak (Aug. 2019). "Serialization in Object-Oriented Programming Languages". In: *Introduction to Data Science and Machine Learning*. IntechOpen. ISBN: 978-1-83880-334-6 978-1-83880-333-9 978-1-83880-371-1. DOI: 10.5772/intechopen.86917. (Visited on 05/20/2026).

Hasabnis, Niranjan (Oct. 2022). "GitRank: A Framework to Rank GitHub Repositories". In: *Proceedings of the 19th International Conference on Mining Software Repositories*. MSR '22. New York, NY, USA: Association for Computing Machinery, pp. 729–731. ISBN: 978-1-4503-9303-4. DOI: 10.1145/3524842.3528519. (Visited on 05/30/2026).

Hopcroft, John E., Rajeev Motwani, and Jeffrey D. Ullman (2007). *Introduction to Automata Theory, Languages, and Computation*. 3. ed. Bosten San Francisco Munich: Pearson. ISBN: 978-0-321-45536-9.

Huth, Marius (Jan. 2026). *Sicherheitsrisiken der Deserialisierung am Beispiel des Qt-Frameworks*.

Hüther, Lorenz, Karsten Sohr, and Christian Maeder (Aug. 2023). *Static Analysis for the Qt Framework*. Final Project Report. Universität Bremen, p. 42.

Internet Relay Chat Protocol (May 1993). Request for Comments RFC 1459. Internet Engineering Task Force. DOI: 10.17487/RFC1459. (Visited on 05/03/2026).

Jelodar, Hamed, Mohammad Meymani, and Roozbeh Razavi-Far (Mar. 2025). *Large Language Models (LLMs) for Source Code Analysis: Applications, Models and Datasets*. DOI: 10.48550/arXiv.2503.17502. arXiv: 2503.17502 [cs.SE]. (Visited on 06/26/2026).

Li, Fengjie, Jiajun Jiang, Dongchi Chen, and Yingfei Xiong (Jan. 2026). *LLM-based Vulnerability Detection at Project Scale: An Empirical Study*. DOI: 10.48550/arXiv.2601.19239. arXiv: 2601.19239 [cs.SE]. (Visited on 06/30/2026).

Liu, Jun, Yuanyuan Xie, Jiwei Yan, Jinhao Huang, Jun Yan, and Jian Zhang (July 2025). *Write Your Own CodeChecker: An Automated Test-Driven Checker Development Approach with LLMs*. DOI: 10.48550/arXiv.2411.06796. arXiv: 2411.06796 [cs.SE]. (Visited on 07/15/2026).

Møller, Anders and Michael I Schwartzbach (Apr. 2025). *Static Program Analysis*. Department of Computer Science, Aarhus University. (Visited on 06/26/2026).

Montgomery, Douglas C. and George C. Runger (Mar. 2010). *Applied Statistics and Probability for Engineers, 5th Edition*. Wiley. ISBN: 978-0-470-05304-1. (Visited on 08/03/2026).

Al-Rubaye, Abduljaleel and Gita Sukthankar (Dec. 2020). "Scoring Popularity in GitHub". In: *2020 International Conference on Computational Science and Computational Intelligence (CSCI)*, pp. 217–223. DOI: 10.1109/CSCI51800.2020.00044. (Visited on 05/30/2026).

Schwartz, Edward J., Thanassis Avgerinos, and David Brumley (May 2010). "All You Ever Wanted to Know about Dynamic Taint Analysis and Forward Symbolic Execution (but Might Have Been Afraid to Ask)". In: *2010 IEEE Symposium on Security and Privacy*, pp. 317–331. DOI: 10.1109/SP.2010.26. (Visited on 07/06/2026).

Shiri Harzevili, Nima, Alvine Boaye Belle, Junjie Wang, Song Wang, Zhen Ming (Jack) Jiang, and Nachiappan Nagappan (Nov. 2024). "A Systematic Literature Review on Automated Software Vulnerability Detection Using Machine Learning". In: *ACM Computing Surveys* 57.3, 55:1–55:36. ISSN: 0360-0300. DOI: 10.1145/3699711. (Visited on 06/26/2026).

Singh, Jaswinder, Anu Gupta, and Preet Kanwal (2024). "The Vital Role of Community in Open Source Software Development: A Framework for Assessment and Ranking". In: *Journal of Software: Evolution and Process* 36.7, e2643. ISSN: 2047-7481. DOI: 10.1002/smr.2643. (Visited on 05/04/2026).

Vaccario, Giacomo, Matúš Medo, Nicolas Wider, and Manuel Sebastian Mariani (Aug. 2017). "Quantifying and Suppressing Ranking Bias in a Large Citation Network". In: *Journal of Informetrics* 11.3, pp. 766–782. ISSN: 1751-1577. DOI: 10.1016/j.joi.2017.05.014. (Visited on 06/05/2026).

Wohlin, Claes, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén (2024). "Analysis and Interpretation". In: *Experimentation in Software Engineering*. Ed. by Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. Berlin, Heidelberg: Springer, pp. 153–184. ISBN: 978-3-662-69306-3. DOI: 10.1007/978-3-662-69306-3_11. (Visited on 06/15/2026).

Yamaguchi, Fabian, Nico Golde, Daniel Arp, and Konrad Rieck (May 2014). "Modeling and Discovering Vulnerabilities with Code Property Graphs". In: *2014 IEEE Symposium on Security and Privacy*. San Jose, CA: IEEE, pp. 590–604. ISBN: 978-1-4799-4686-0. DOI: 10.1109/SP.2014.44. (Visited on 06/26/2026).

Yang, Chenyuan, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang (Oct. 2025). "KNighter: Transforming Static Analysis with LLM-Synthesized Checkers". In: *Proceedings of the*

ACM SIGOPS 31st Symposium on Operating Systems Principles, pp. 655–669. DOI: 10.1145/3731569.3764827. arXiv: 2503.09002 [cs.SE]. (Visited on 07/14/2026).

Yi, Kwangkeun and Xavier Rival (Feb. 2020). *Introduction to Static Analysis: An Abstract Interpretation Perspective*. Cambridge, MA, USA: MIT Press. ISBN: 978-0-262-35665-7.

Zhang, Yu, Min Wang, Morteza Saberi, and Elizabeth Chang (July 2022). "Analysing Academic Paper Ranking Algorithms Using Test Data and Benchmarks: An Investigation". In: *Scientometrics* 127.7, pp. 4045–4074. ISSN: 1588-2861. DOI: 10.1007/s11192-022-04429-z. (Visited on 06/05/2026).

Lists

List of Figures

3.1 Prevalence Analysis Pipeline. 12
3.2 Prevalence Analysis Results. 13
3.3 Score Distribution of QDataStream Projects. 14
3.4 Age Distribution Comparison. 15
3.5 QDataStream Usage Rate per Year. 15
3.6 Relationship Between Score and Age in QDataStream Projects. 16

4.1 Internal Representations. 22
4.2 Vulnerability Analysis Approach. 27

List of Tables

3.1 Source Code Hosting Providers. 7
3.2 Prevalence Analysis Results. 13
3.3 Top 10 QDataStream Projects Ranked by Score. 13

4.1 Verification results. 31
4.2 Results of the static analysis. 32

List of Listings

2.1 Write Binary Data to a Stream. 5
2.2 Read Binary Data From a Stream. 5
2.3 Quassel 0.12.4 – datastreampeer.cpp. 6
2.4 Quassel 0.12.5 – datastreampeer.cpp. 6

3.1 Verifying Qt Projects. 11

4.1 Buffer Overflow Example. 20
4.2 W1 Check: Identifying QDataStream Bound Sites. 28
4.3 W1 Check: Classifying Sources. 28
4.4 W1 Check: Taint Query on Sources Classified Unknown. 29
4.5 W1 Check: Identifying Sinks. 29
4.6 W1 Check: Finding Untrusted Reads. 30
4.7 QtSingleApplication – qtlocalpeer.cpp. 33
4.8 W3 Exploit for Krita and Tiled. 34

List of Abbreviations

AI Artificial Intelligence

API Application Programming Interface

AST Abstract Syntax Tree

CBOR Concise Binary Object Representation

CFG Control-Flow Graph

CPG Code Property Graph

CPU Central Processing Unit

CVE Common Vulnerabilities and Exposures

CVSS Common Vulnerability Scoring System

CWE Common Weakness Enumeration

IRC Internet Relay Chat

JSON JavaScript Object Notation

LLM Large Language Model

NVD National Vulnerability Database

PDG Program Dependence Graph

QML Qt Meta-Object Language

RQ Research Question

SQL Structured Query Language

TCP Transmission Control Protocol

WTPS Weighted Total Popularity Score

XML Extensible Markup Language